

目录

第 1 章 Arduino 介绍	1
1.1 什么是 Arduino	2
1.1.1 微控制器.....	2
1.1.2 开源硬件.....	3
1.1.3 探究 Arduino 架构	3
1.2 Arduino 家族介绍	4
1.2.1 Arduino 的历史	4
1.2.2 Arduino 型号	5
1.2.3 熟悉 Arduino Uno.....	6
1.3 Arduino 扩展板	6
1.4 本章小结.....	7
第 2 章 建立 Arduino 编程环境	8
2.1 微控制器内部探秘.....	8
2.1.1 揭秘 ATmega 微控制器	8
2.1.2 微控制器编程.....	9
2.2 超越机器代码.....	10
2.2.1 编写汇编代码.....	10
2.2.2 C 语言使开发更容易	10
2.3 编写 Arduino 程序	11
2.3.1 Atmel C 代码库	11
2.3.2 使用 Arduino 编程工具	11
2.4 Arduino 配置	12
2.4.1 配置环境.....	12
2.4.2 测试一下环境是否配置成功.....	15
2.5 本章小结.....	17
第 3 章 编写 Arduino 程序	18
3.1 编写 Arduino 程序	18
3.1.1 Arduino 程序构成	18
3.1.2 包含库.....	19
3.2 编写第一个程序.....	19
3.2.1 使用编辑器.....	19
3.2.2 程序编译.....	20
3.2.3 上传程序.....	21
3.2.4 运行程序.....	21
3.3 连接硬件电路.....	22
3.3.1 使用扩展插槽.....	22
3.3.2 使用面包板.....	23
3.3.3 使用原型扩展板.....	24
3.3.4 使用印刷电路板.....	24
3.4 小结.....	25
第 4 章 实验项目	26

4.1 函数信号发生器的设计、组装与调试研究.....	26
4.2 电容测量仪的设计、制作与调试.....	32
4.3 温度检测显示控制器的设计与制作.....	36
4.4 倒计时定时器的设计、制作与调试.....	42
4.5 Arduino 实时音频处理实验	49
4.6 多功能数字时钟的设计与制作.....	55
4.7 Arduino 万用表的设计与制作	67
4.8 Arduino 数字示波器的设计与制作.....	71

第 1 章 Arduino 介绍

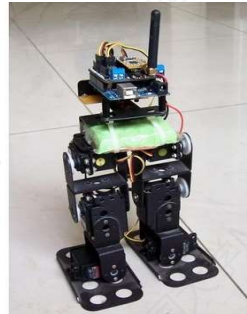
本章将要学习：

- Arduino 到底是什么
- Arduino 微控制器家族概述
- 在 Arduino 上可以运行什么样的程序

近年来，Arduino 已经在电子设计圈内掀起了一阵风暴。使用开源硬件平台开发微控制器应用，点燃了业余电子爱好者和专业人士的热情，以下是 Arduino 控制器和开发应用实例。



Arduino 控制器



双足机器人



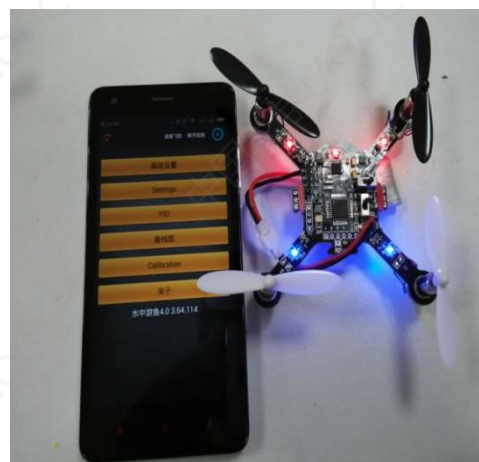
虫虫机器人



智能小车



Arduino 控制器



手机控制四旋翼飞机



3D 打印机



3D 打印产品

1.1 什么是 Arduino

Arduino 是一款开源硬件平台，可以用它接收数字或模拟信号，也可以发送数字或模拟信号来控制外部设备。

在开始介绍 Arduino 的编程知识前，先了解一下 Arduino 的背景。

1.1.1 微控制器

似乎每件东西里都嵌入了一个小型计算机。从烤箱到电视，难以找到一种设备没有任何类型的计算机系统控制它。尽管如此，嵌入到烤箱中的处理器芯片和计算机中的处理器芯片还是有很大不同。嵌入到家用电器中的处理器芯片的计算能力不需要像计算机中的那么强劲。它们只需要监控一些模拟或者数字信号（比如时间或温度），并且能控制部分设备（比如烤箱中的加热部件或者电视机上的频道设置）。

因为嵌入式计算机要求更低的功耗，所以计算机上的昂贵而高功耗的芯片就不太适合嵌入式场合。取而代之的是便宜得多的低功耗处理器芯片，这就是微控制器。

微控制器是一种处理能力很弱的芯片，一般用在自动控制领域。微控制器的设计理念是尽量简化，而不是在芯片中集成太多的功能和部件。

因为微控制器不需要非常强大的处理单元，所以在芯片中留下了很多空间可以集成其他功能，这些功能在通用计算机芯片中一般是用外部电路实现的。比如，大多数微控制器内部都集成了存储器和 I/O 端口。它们提供了访问微控制器所有功能的简单接口，使得微控制器和外部电路更容易集成。图 1.1 是微控制器的基本结构框图。

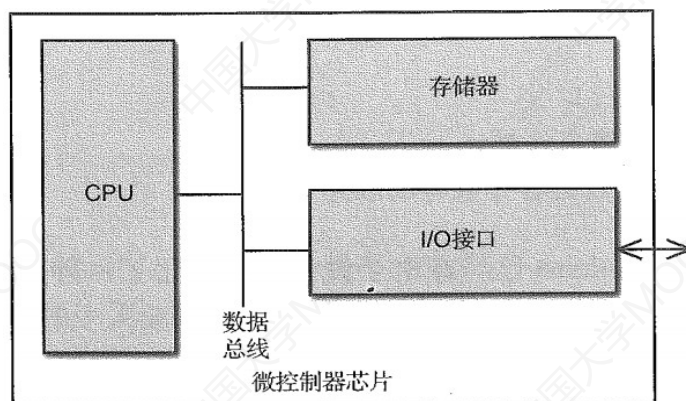


图 1.1 微控制器框图

通过将处理单元、存储器和 I/O 端口集成到一块芯片中，微控制器为设计人员提供了简单的接口，以方便应用到工程项目中。

1.1.2 开源硬件

如果你听说过 Linux 操作系统，那么可能会比较熟悉开源软件的概念。在开源软件社区，开发者们向公众发布程序代码，任何人都可以对代码修改和错误修正提出建议。结果就是丰富了开源软件的功能，并且 bug 更少。

开源硬件的“开源”概念和开源软件的基本一致，区别仅在于开源的对象是硬件而不是软件代码。在开源硬件项目中，硬件设计对公众开放，允许自由使用和修改。

Arduino 项目开发者设计了一个完整的微控制器系统，使用标准接口和外部设备交互。项目架构和开发计划已经作为开源项目向公众开放，允许任何人在自己的项目中自由使用，甚至修改，而不会触犯任何专利保护法案。任何人可在 arduino.cc 网站上免费下载电路图（以及程序开发软件），自己 DIY 一个 Arduino，也能在网上购得各种形式的 Arduino 兼容板（以“duino”或“ino”名称结尾，例如：Freeduino、Japanino、Zigduino 等）。

1.1.3 探究 Arduino 架构

Arduino 的开发人员在单板上构建了完全独立的微控制器系统，可以将其插入到任何嵌入式设备中。Arduino 项目的核心是其中的微控制器芯片。

ATmega AVR 微控制器家族

Arduino 开发人员选择 Atmel ATmega AVR 微控制器家族作为 Arduino 芯片。ATmega AVR 微控制器是一种 8 位处理器芯片，具有如下特性：

- 内置闪存（Flash），用于存放代码；
- 静态随机访问存储器（SRAM）用于保存运行时的数据；
- 电可擦除只读存储器（EEPROM）用于保存长期数据；
- 数字 I/O 端口；
- 模拟/数字转换器（ADC），用于将模拟输入信号转换成数字信号。

ATmega AVR 是一种单芯片的微控制器系统。Arduino 开发者可以用它和最少的外部电路构建出很小的电路板。

Arduino 系列使用了几种不同的 ATmega 处理器，如表 1.1 所示。

表 1.1 ATmega 处理器家族

处理器	Flash	SRAM	EEPROM
ATmega48	4 KB	512 B	256 B
ATmega88	8 KB	1 KB	512 B
ATmega168	16 KB	1 KB	512 B
ATmega328	32 KB	2 KB	1 KB
ATmega32u4	32 KB	2.5 KB	1 KB

与计算机的存储器容量（一般以 GB 为单位计算）相比，系列微控制器只有非常有限的存储空间。但是为微控制器开发的程序一般都相当简单，所以基本上不用担心存储器空间的限制问题。

Arduino 布局

Arduino 的另一个关键问题是它如何与外部设备连接。Arduino 开发者设计了一种标准接口，使得其他开发者可以很容易地将 Arduino 集成到他们自己的项目中。（这也是开源硬件项目的通用思路之一。）

Arduino 使用扩展插槽将处理器的 I/O 引脚向外扩展。图 1.2 展示了 Arduino UNO 的布局。

可以通过扩展插槽访问微控制器的所有接口。图中底部的扩展插槽包含了模拟输入信号、电源引脚和接地。顶部的扩展插槽包含了数字 I/O 信号。扩展插槽的标准化使得开发人员可以方便地设计 Arduino 的扩展板。



图 1.2 Arduino UNO 布局。

1.2 Arduino 家族介绍

Arduino 开发板不止一种型号，本节将介绍 Arduino 的历史，以及衍生出的多种不同版本 Arduino。

1.2.1 Arduino 的历史

Arduino 并不是由大型电子公司设计的，甚至也不是由计算机科班生研发出来的。它最初只是为了满足一群学生和老师的艺术设计项目的需要而设计的。

现代艺术项目经常需要使不同的物体同步运动，这需要精确的自动化操作，换句话说，这就需要某种类型的微控制器系统。因为大多数艺术专业的学生没有工程底子，所以购买微控制器芯片和设计电路对他们来说是个很大的挑战。一群在意大利艾维里互动设计学院的学生设计了一个项目，用于帮助那些需要编写自动控制程序的艺术专业学生降低编程难度。

随后诞生了一个称为 Wiring 项目，该项目包括一款标准微控制器电路板和一個标准编程环境（称为 Processing）。2003 年，Wiring 和 Processing 项目得到一定程度的认可，但是

对于学生来说还是有些昂贵。

2005 年，Massimo Banzi 和 David Cuartielles 领导的一组开发者建立了 Arduino 项目。Arduino 建立在 Wiring 项目的基础之上，造价却低廉得多。

考虑到非技术人员也能使用，Arduino 系统的每个部分都尽量简化设计。硬件接口设计容错性较好。举例来说，假如将错误的引脚连接到了错误的端口上，一般不会毁坏板子。即使微控制器芯片损坏，Arduino 的设计也允许只更换损毁的微控制器芯片，而不用购买全新的 Arduino 电路板。

同样，考虑到非专业程序设计人员，Arduino 的软件设计也尽量降低门槛。为迎合艺术领域的学生，程序被称为素描（sketches），保存程序（素描）的文件夹被形象地称为写生簿（sketchbooks）。

1.2.2 Arduino 型号

开源硬件的一个设计理念是为开发者提供足够多的选项，让他们自己决定最适合自己的项目的 Arduino 型号。下面各图展示了一些常用 Arduino 型号。



Arduino UNO



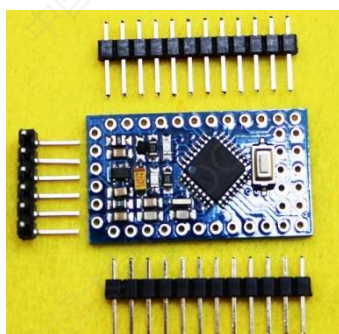
Arduino DUE



Arduino Mega



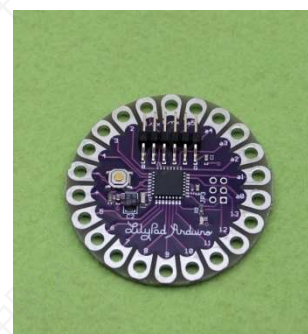
Arduino Leonardo



Arduino mini



Arduino nano



Arduino LilyPad

Arduino Uno

Arduino 家族中的主力是 Uno。它是最流行的 Arduino 分支，通过标准的 Arduino 接口

提供了 ATmega AVR 微控制器的基本功能。

Arduino Uno 的尺寸和布局已经成为大多数 Arduino 分支的参考标准，所以几乎所有 Arduino 扩展板都能适配 Arduino Uno 的扩展插槽。

1.2.3 熟悉 Arduino Uno

Arduino Uno 通常被用在需要简单微控制器的场合，其提供了如下接口：

- 14 路数字 I/O 接口
- 6 路模拟输入信号
- 6 路 PWM 信号
- 1 个 I²C 接口
- 1 个 SPI 接口
- 1 个 UART 接口，通过 USB 接口转换

如图 1.2 所示，Arduino Uno 电路板采用了标准的 Arduino 扩展插槽。

底部一横排的插槽中，右边是模拟输入信号，左边是电源和接地。顶部是数字 I/O 插槽。数字 I/O 插槽有两种用途。除了用作数字输入/输出信号，部分引脚还可被复用，具有其他功能：

- 数字插槽 0 和 1 也分别被用作 UART 接口的 RX 和 TX 信号；
- 数字插槽 3、5、6、9、10 和 11 被用作 PWM 输出信号；
- 最左侧的 4 个插槽（10、11、12、13）用作 SPI 接口信号。

除了扩展插槽，Arduino Uno 还提供标准 USB 接口用于连接计算机。USB 信号被转换成 ATmega 微控制器上的 UART 串口信号，以实现计算机与 Arduino 的通信。这也是将程序从计算机下载到 Arduino 的通道。还可以使用该 USB 串口查看微控制器的打印输出，这是调试程序时的便利工具。

Arduino Uno 还有 4 个内置 LED，可以帮助观察 Arduino 的工作情况：

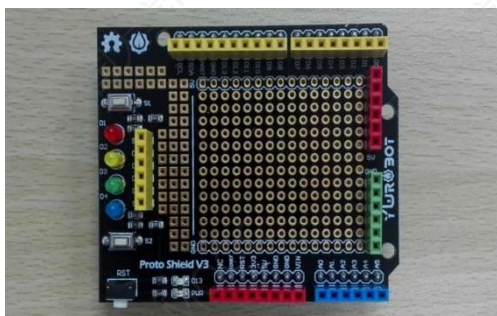
- 绿色 LED（标注为 ON）亮起指示 Arduino 已接通电源；
- 两个黄色分别标注 RX 和 TX 闪烁指示接口正在收发数据；
- 黄色（标注为 L）连接到数字输出信号插槽 13，可以编程对其进行控制。

Uno R3 上新增的部件是电路板左上角的复位按键。按下复位按键将使 ATmega 微控制器立即重启，重新将程序加载到内存，并从程序开头的第一条语句开始执行。

1.3 Arduino 扩展板

在项目中使用开源硬件的好处在于有很多其他的开发者已经解决了你可能遇到的同样问题，并且愿意分享他们的方案。Arduino 的标准接口为开发人员提供了设计外部电路的通用方案。

这些外部电路被称为扩展板，下图列举一些扩展板，分别是原型扩展板、传感器扩展板、游戏手柄扩展板和舵机扩展板。



Arduino Proto Shield



Arduino Sensor Shield



Arduino JoyStick Shield



Arduino UNO+舵机扩展板

1.4 本章小结

本章涵盖 Arduino 的基本概念，以及在项目中如何使用 Arduino。

介绍了什么是微控制器，以及如何使用微控制器接收外部数字和模拟信号，并向外部设备输出数字信号。

讨论如何使用标准接口配合 ATmega 微控制器收发信号。

介绍了用 Arduino 扩展板为 Arduino 增强功能。

第 2 章 建立 Arduino 编程环境

本章将要学习

- 微控制器如何工作
- 微控制器程序如何运行
- 什么是 Arduino 编程语言
- 如何下载和安装 Arduino

仅拥有 Arduino 硬件并不会让项目运行起来。除了硬件，还需要编写程序告诉 Arduino 硬件要做什么工作。

本章介绍微控制器基本工作原理，即它是如何读取和运行程序，以及如何为 Arduino 建立编程开发环境。

2.1 微控制器内部探秘

用 Arduino 编写程序前，了解它的内部工作原理会很有帮助。所有 Arduino 电路板的核心是 ATmega AVR 系列微控制器。Arduino 程序都运行在 ATmega 微控制器上。本节将介绍 ATmega 微控制器的内部结构，展示各个部件以及它们如何协作。

2.1.1 揭秘 ATmega 微控制器

微控制器的三个基本部件是：

- CPU
- 存储器
- 输入/输出（I/O）接口

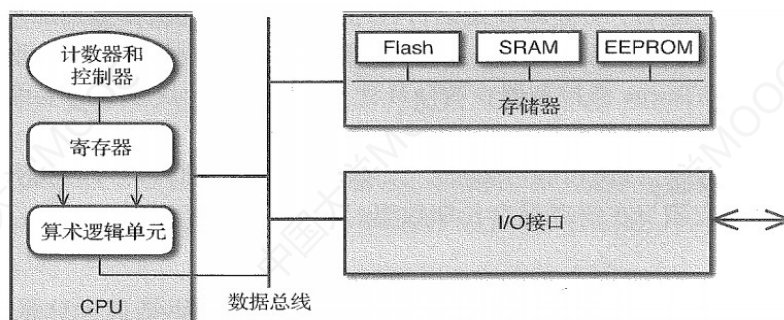


图 2.1 ATmega 微控制器内部结构

每个部件的细节如何，它们是如何相互配合工作的？

CPU

微控制器内部的 CPU 单元和计算机上的 CPU 基本相同，只是尺寸更小而已。在大多数计算机中，CPU 由几个通用部件组成：

- 算术逻辑单元（ALU），执行数学和逻辑运算；
- 数据寄存器，保存 ALU 执行运算所需的数据；
- 状态寄存器，保存最近执行指令的状态信息；
- 程序计数器，保存下一条指令地址；

- 栈指针，跟踪内存中临时数据的存放位置；
- 控制器，装载程序代码和数据到寄存器、ALU 和指针寄存器。

所有的 CPU 部件的工作都依赖于外部提供给微控制器的时钟信号。在每个时钟周期内，CPU 完成一个操作。时钟频率决定了 CPU 运行指令的速度。

例如，Arduino Uno 使用 16 MHz 时钟，也就是每秒提供 1600 万个时钟脉冲。大多数 ATmega 微控制器的指令在 1 到 2 个时钟周期内执行完成，这使得 Arduino 对输入信号的采样非常快。

存储器

Atmel ATmega 微控制器家族使用哈佛体系结构。该体系结构将计算机存储器分成两大部分：一部分用于保存程序代码，另一部分用于保存数据。

ATmega 微控制器分别使用两种类型的存储器来实现哈佛结构：

闪存，保存程序代码；

静态随机访问存储器（SRAM），保存程序运行时数据。

闪存存在微控制器断电后仍可保存数据。这就是 Arduino 保存程序代码的方法。对 Arduino 重新上电后，会立即开始运行程序，而不需要每次都重新加载。

SRAM 在断电后无法保存数据，任何保存在 SRAM 中的数据都将丢失。为了弥补这个缺点，ATmega 微控制器在 SRAM 之外，还使用一块独立的电可擦写只读存储器 EEPROM 来保存数据。

EEPROM 在掉电后仍可保存数据，除非手动清除它们。它的运行速度不如 SRAM 快，所以应该只在需要保存长期数据时才使用 EEPROM。

输入输出接口

I/O 接口实现了微控制器与外部的通信。ATmega 可以通过 I/O 接口读取外部设备的模拟或数字输入信号，并且输出数字信号。

ATmega 控制器包含内置的模拟/数字转换器（ADC），它将模拟输入信号转换为可以用程序读取的数字信号。另外，还内置了数字信号接口的控制电路，通过设置可以将同一个引脚既作为输入也作为输出接口。

微控制器中的所有部件协同工作，CPU 是总的控制者。下一节将进一步介绍 CPU 如何工作。

2.1.2 微控制器编程

为 CPU 编写程序可以控制它的运行。CPU 读取程序指令，一次完成一条指令操作。每种类型的都有自己能够解释的指令，统称为指令集。

指令集的意义是告诉 CPU 检索何种数据以及如何处理数据。这也正是程序所要做的。诸如读取数据、数学运算、输出数据等操作都在指令集中有相应定义。

因为 CPU 只能读写二进制数据，所以指令都是 1 个或 2 个字节的二进制命令。这些命令称为机器码（通常也称为操作码，或者简称为 opcode）。指令由指令码和数据组成。

ATmega AVR 指令长 16 位（2 字节）。例如，将寄存器 R1 和 R2 中的数据相加，结果保存在 R1 中的指令为

```
0000 1100 0001 0010
```

当 CPU 处理该指令时，它指示 CPU 从寄存器取出数据，在 ALU 中把数据相加，然后把结果放回 R1。

2.2 超越机器代码

微控制器直接使用机器码处理数据，但是机器码不容易阅读和理解。为解决这个问题，开发者创造了编写程序的其他方法。本节将讨论这些方法，并解释如何使用它们进行 Arduino 编程。

2.2.1 编写汇编代码

汇编语言和机器码非常相近，它为每条机器指令分配了一个文本助记符。文本助记符通常是代表指令功能的短字符中（比如 ADD 两数相加）。这种方法让编写基本的微控制器程序容易了很多。将寄存器 R1 和 R2 的值相加，并将结果保存到 R1 中的机器码如下：

```
0000 1100 0001 0010
```

相应的汇编语言如下：`add r1, r2`

这样的表达形式看起来更有意义，编写代码时更易于记忆。ATmega AVR 的每条机器指令都有相应的汇编语言助记符。在 AVR 指令集手册中有详细清单。

ATmega AVR 系列微控制器有 282 条指令，相应地有 282 条汇编助记符。这些指令可以分为 6 大类：

- 从内存装载数据到寄存器
- 寄存器数据的数学运算
- 寄存器间数据比较
- 复制寄存器值到内存
- 分支跳转
- 与数字 I/O 接口交互

对每个寄存器、每种数学运算、每种寄存器和内存间的数据复制方法，都有单独的指令。虽然 Atmel AVR 汇编语言在机器语言基础上取得了进步，但是用汇编语言创造大型复杂程序仍然很困难，这种工作只能由经验丰富的高级程序员完成。幸运的是，在 Arduino 上还有更容易的编程方法。

2.2.2 C 语言使开发更容易

即使使用了汇编语言，在 282 条指令中记住某种操作到底该用哪条指令仍然是个很大的挑战。而更高层次的语言使得编程工作更加容易。

高级编程语言将微控制器的内部工作原理和程序代码逻辑分离。高级编程语言为程序员隐藏了大多数的 CPU 内部细节，使得程序员可以专注于程序逻辑。

例如，在高级编程语言中，可以为变量赋值，而编译器将变量名转换成合适的内存地址或者寄存器。在高级编程语言中，两数相加，再将结果存回同一地址，可以编写如下代码：

```
value1 = value1 + value2;
```

这条语句比机器语言的版本更容易理解。

使用高级语言的关键是必须有相应的编译器软件，它将对用户友好的程序代码翻译成能在微控制器上运行的机器语言代码。这个工作由编译器和代码库共同完成。

编译器读取高级语言代码，转换成机器语言代码。因为目标对象是机器语言，所以编译器和 CPU 相关的。不同的 CPU 要求有不同的编译器。除了能将高级语言转换成机器代码，

大多数编译器都支持一组通用函数,使程序员编写代码 更加容易。这被称为代码库。同样,不同的 CPU 有不同的代码库。

现在有很多不同的高级编程语言,但是开发者选择流行的 C 语言作为 AVR 微控制器的编程语言。他们为微控制器发布了一款编译器,以及可以访问微控制器上数字和模拟端口的库函数。

当 Arduino 项目开发人员选择 ATmega 微控制器作为处理器时,也希望项目编码对于非专业程序员更容易。为了达到这个目的,在 Atmel C 语言编译器和代码库基础上,构建了以另一个适用于 Arduino 开发环境的代码库。

2.3 编写 Arduino 程序

在 Arduino 开发环境中有两个主要的 C 语言库可供使用:

Atmel 代码库

Arduino 代码库

Atmel 代码库是为 ATmega 微控制器建立的通用代码库,Arduino 代码库是特别为使用 Arduino 的非专业程序员设计的库。

2.3.1 Atmel C 代码库

Atmel C 库环境包括两个基本的开发包:

一个命令行编译环境

一个图形化编译环境

AVR Libc 项目为 ATmega AVR 微控制器建立了完整的 C 语言命令行编译器和代码库。它包含三个核心部分:

- avr-gcc 编译器,用于把 C 语言翻译成机器代码;
- avr-libc,为 AVR 微控制器提供了 C 语言开发库;
- avr-binutils,为 C 语言编程提供了额外的工具。

avr-gcc 使用开源的 GNU C 编译器 (gcc),针对 ATmega 微控制器进行了定制,以生成 AVR 机器代码。这就是将 C 程序翻译成机器代码的方式。

近来几乎所有的应用程序都使用了图形化界面,也包括编译器。Atmel 开发者已经发布了一款完整的集成化开发环境(IDE),为微控制器提供了编写和编译 C 程序的图形化窗口环境,名为 Atmel Studio。

虽然 Atmel C 代码库简化了 ATmega 系列微控制器的编程工作,它仍被认为只适合高级程序员使用。取而代之的是,大多数 Arduino 初学者(甚至有些高级开发人员)使用 Arduino 项目提供的 Arduino 编程工具。

2.3.2 使用 Arduino 编程工具

Atmel C 库的一个缺点是它使用常规代码与 ATmega 微控制器打交道。Arduino 硬件平台的主要优点是可以方便地访问 ATmega 微控制器的硬件功能,但是使用 ATmega 库将丧失这个优点。Arduino 开发团队解决了这个问题,他们创造了 Arduino 编程语言。

Arduino 编程语言

Arduino 开发者创造了包含很多额外功能的 C 代码库，它可以比 Atmel 库更容易地访问 Arduino 的硬件特性。

例如，向数字输出引脚发送信号，只要使用函数 `digitalWrite()`；从模拟输入引脚读取信号，只要使用函数 `analogRead()`。这就使在 Arduino 上编写代码容易很多。

Arduino 开发人员将定制化的 C 库和 Atmel avr-gcc 编译器放到一个包中发布，称为 Arduino IDE。Arduino IDE 和 Atmel Studio 一样，也是图形化界面，但是没有 Atmel Studio 那么复杂。图 2.2 展示了基本的 Arduino IDE 窗口。

这个 IDE 包含了编辑器、编译器和上传程序到 Arduino 上的工具。



图 2.2 Arduino IDE 窗口

Arduino 扩展板开发库

除了 Arduino C 代码库，Arduino 团队还为所有通用 Arduino 扩展板开发了相应的库。这样就可以轻松地将扩展板的功能集成到自己的程序中，而不用编写复杂的代码。

Arduino IDE 默认包含所有流行的扩展板库。有了 Arduino IDE 和扩展板库，就可以创造任何你想要的 Arduino 项目。

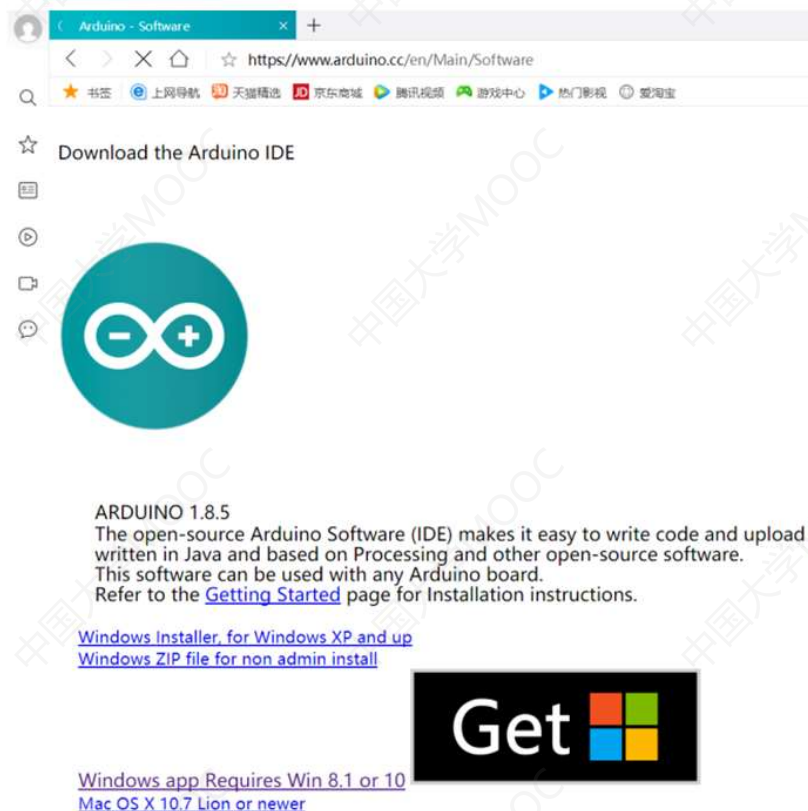
2.4 Arduino 配置

下面说明如何在 win10 系统上配置 Arduino 环境

2.4.1 配置环境

首先第一步，先从官网下载 Arduino IDE。

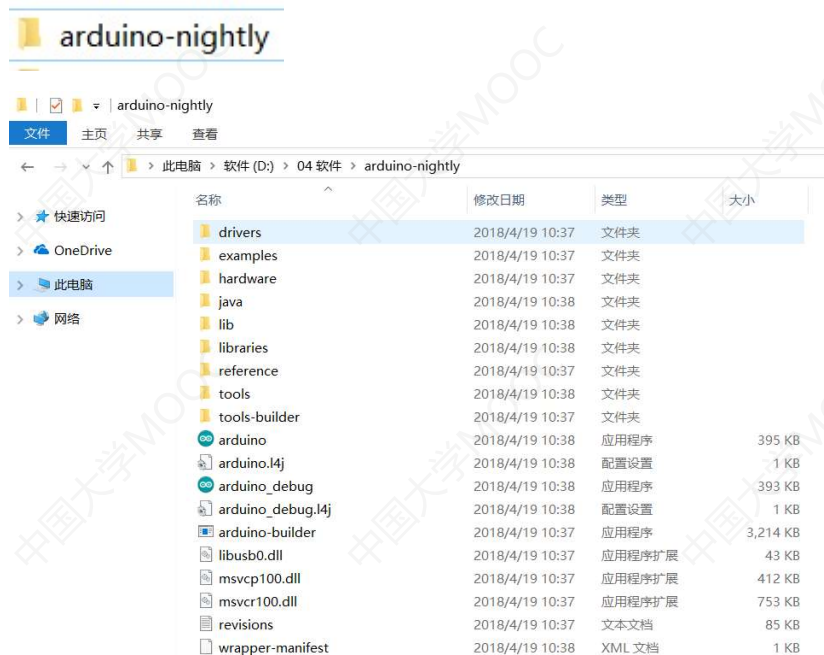




对于 win10 系统，下载对应的 Arduino IDE （Windows app Requires Win8.1 or 10），获取压缩文件



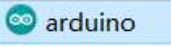
解压至相应文件夹

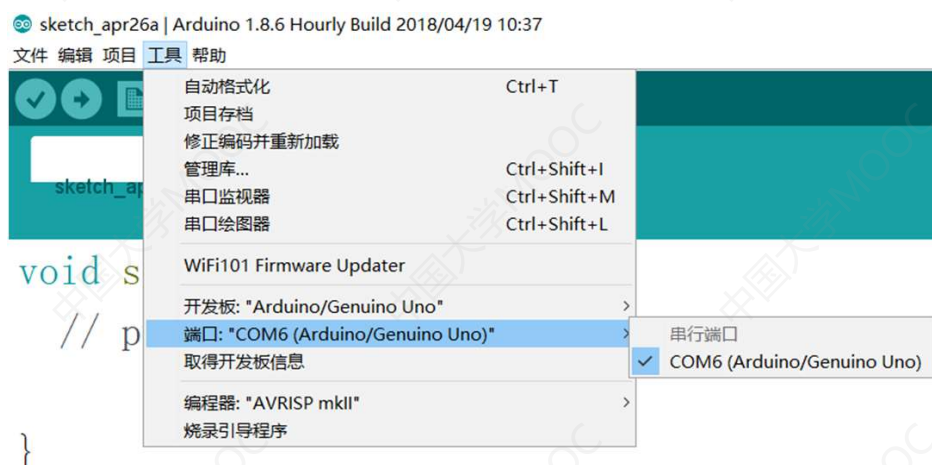


第二步，将 Arduino Uno 板通过 USB 线连接到电脑上，稍等一会儿，会自动安装驱动。

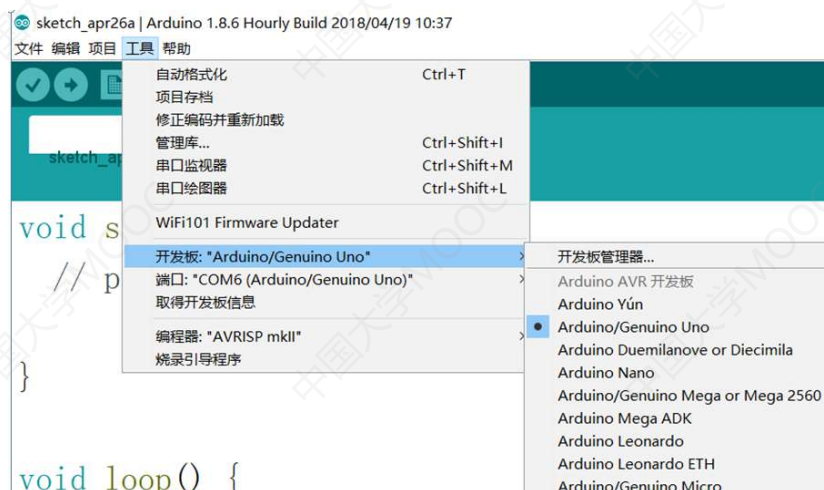
第三步，打开“设备管理器”，找到“端口”，记住端口号，如下图：



第四步，运行  打开 Arduino IDE ，按下图操作：



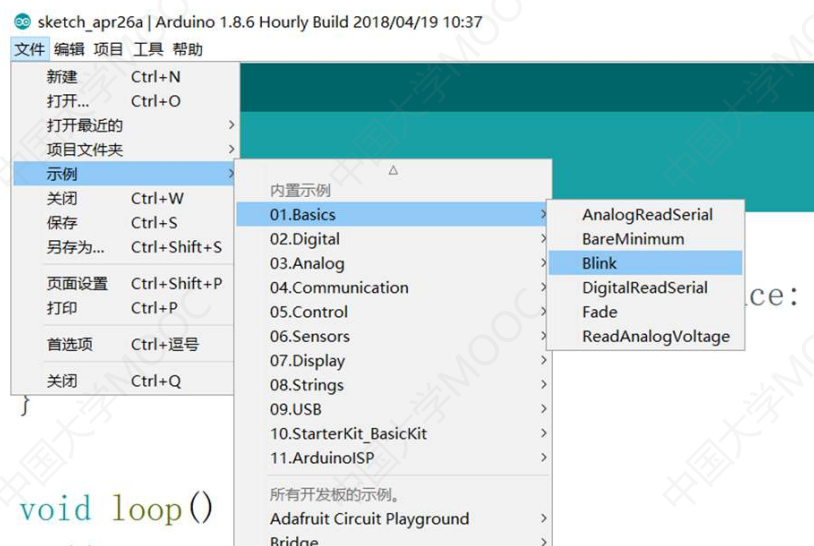
第五步，如下图：



到这里，我们的 Arduino 就可以用了。

2.4.2 测试一下环境是否配置成功

第一步，按下图在 Arduino IDE 中选择 Blink 程序，并做如下操作：





编译后上传



第二步, Arduino 板上的“接脚 13”旁的灯一闪一闪的, 说明我们的代码确实已经上传到 Arduino 板上了, 所以我们的环境配置成功了。



2.5 本章小结

本章介绍了如何为 Arduino 编写程序。Arduino 的核心是 ATmega 微控制器，所以代码必须能在该芯片上运行。ATmega 微控制器使用机器语言处理数据，并与输入/输出端口交互。可以使用 Atmel C 语言开发库简化程序开发。同时，Arduino 开发团队对 Atmel C 语言进行了扩展，提供了自己的编程环境和开发库。这使得为 Arduino 开发程序变得更加容易。可以从 Arduino 网站上 Arduino IDE 软件包的各种版本，它们支持 Windows，OS X 和 Linux 三种平台。

第 3 章 编写 Arduino 程序

本章将要学习：

- 编写 Arduino 程序
- 编译和运行 Arduino 程序
- 在 Arduino 上连接电路板

已经知道 Arduino 是什么，以及如何使用 Arduino IDE 编写程序，现在到了亲自编写第一个 Arduino 程序的时候了。本章介绍如何使用 Arduino 创建、编译和上传程序，以及如何把外部电路连接到 Arduino 加以完成整个工程。

3.1 编写 Arduino 程序

建立 Arduino 开发环境后，便可以着手开发项目了。本节涵盖了编写和运行程序所需的基本知识。

3.1.1 Arduino 程序构成

使用 Arduino 时，必须遵循特定的编码规范。这个编码规范和标准 C 语言有一点小区别。

在标准 C 语言中，通常用 main 函数定义程序的入口。当 CPU 开始运行程序时，从 Arduino 函数开始执行。

但是，Arduino 程序中却没有 main 函数。预先装载到 Arduino 上的 bootloader 作为程序的 main 函数。Arduino 首先执行 bootloader，bootloader 再调用用户编写的应用程序。bootloader 会到程序中查找两个特殊的函数：

```
setup  
loop
```

上电时，Arduino bootloader 调用就 setup（）函数作为首先执行的函数。setup 函数中的代码只执行一次；然后 bootloader 调用 loop 函数。

setup 函数定义如下：

```
void setup() {  
  code lines  
}
```

通常把启动代码放在 setup 函数中。

bootloader 调用完毕 setup 函数后，就开始反复调用 loop 函数，直到 Arduino 断电。loop 函数的格式和 setup 函数相同：

```
void loop() {  
  code lines  
}
```

应用程序主体在 loop 函数中。在这里程序读取传感器，并向外部发送信号。setup 函数是初始化输入/输出引脚的好地方，这样，在 loop 函数运行时，引脚状态已经就绪，在 loop 函数中可以使用它们了。

3.1.2 包含库

依据 Arduino 程序的复杂程度，可能需要使用外部库文件中的函数。如果要使用外部库函数，首先要在 Arduino 程序的开头用 `#include` 语句包含它们：

`#include <library>`

`#include` 语句通常是程序的第一行，放在其他所有代码之前。

如果使用标准的 Arduino 扩展板，其相关的库可能已经包含在 Arduino IDE 安装包中了。在菜单栏中选择 Sketch>Import Library 选项，然后选择使用的扩展板。

Arduino IDE 会自动添加扩展板所需的 `#include` 语句。例如，如果选择了以太网扩展板，右面几行将自动导入到程序中：

```
#include <Dhcp.h>
#include <Dns.h>
#include <Ethernet.h>
#include <EthernetClient.h>
#include <EthernetServer.h>
#include <EthernetUdp.h>
#include <util.h>
```

3.2 编写第一个程序

现在已经掌握了 Arduino 程序的基本知识，让我们开始编写一个简单的程序，从而对整个开发流程有一个直观认识。

3.2.1 使用编辑器

打开 Arduino IDE 后，编辑窗口将打开一个新的程序。新程序的名称在其编辑器选项卡的顶部，格式如下：

`sketch_mmmddx`

mmm 是月份的英文字母缩写，dd 是用两个数字表示的天数，x 是用于区分同一天 创建的不同程序的字母。

在编辑器中输入代码时，编辑器会对程序的不同部分自动着色，比如函数名显示为棕黄色，字符串显示为蓝色。这样会让编程开发者更容易发现语法错误，在调试程序时非常有用。

现在开始编码吧。表 3.1 展示了程序 Sketch0301 代码。请在 Arduino 编辑器中输入以下代码。

程序的基本功能是让 Arduino 电路板上连接到数字端口 13 的标注为 L 的 LED 每秒钟闪烁一次，并在 Arduino 串口设备上输出信息，对 LED 的每次闪烁进行计数。

在编辑器中输入代码后，从菜单栏中选择 File>Save As，将程序保存为文件 sketch0301。

```

sketch0301
int counter = 0;
int pin = 13;

void setup() {
  Serial.begin(9600);
  pinMode(pin, OUTPUT);
  digitalWrite(pin, LOW);
}

void loop() {
  counter = counter + 1;
  digitalWrite(pin, HIGH);
  Serial.print("Blink #");
  Serial.println(counter);
  delay(1000);
  digitalWrite(pin, LOW);
  delay(1000);
}

```

3.2.2 程序编译

下一步是将程序编译成 Arduino 可以运行的机器语言代码。单击工具栏上的 Verify 图标(勾形图标),或者选择菜单栏中的 Sketch>Verify/Compile 选项。图 3.1 展示了成功编译后的结果。

```

delay(1000);
}

```

编译完成。
项目使用了 2258 字节,占用了 (7%) 程序存储空间。最大为 32256 字节。
全局变量使用了198字节, (9%) 的动态内存, 余留1850字节局部变量。最大为2048字节。

图 3.1 消息区编译成功的消息

如图 3.1 所示,应该可以在消息区看到显示程序已经编译成功的消息,在控制台窗口看到最终编译结果占用的空间大小。

sketch0301 | Arduino 1.8.6 Hourly Build 2018/04/19 10:37
文件 编辑 项目 工具 帮助

```

sketch0301
int counter1 = 0;
int pin = 13;

void setup() {
  Serial.begin(9600);
  pinMode(pin, OUTPUT);
  digitalWrite(pin, LOW);
}

void loop() {
  counter = counter + 1;
  digitalWrite(pin, HIGH);
  Serial.print("Blink #");
  Serial.println(counter);
  delay(1000);
  digitalWrite(pin, LOW);
  delay(1000);
}

'counter' was not declared in this scope
exit status 1
'counter' was not declared in this scope

```

图 3.2 错误信息显示在控制台窗口

Arduino IDE 同时会将出错的代码行高亮显示,让开发者更快地定位错误。更详细的如

果程序中有任何拼写错误，导致编译失败，则会在消息区显示出错信息，如图 3.2 所示。错误信息显示在控制台窗口中。

程序编译通过后，下一步就是上传到 Arduino 电路板运行了。

3.2.3 上传程序

单击工具栏上的 Upload 图标（向右的箭头），或者选择 File>Upload 菜单，就可完成程序上传。开始上传之前，Arduino IDE 会重新编译程序。当对程序做小的修改时，这会显得很方便；只要单击一次，就可以完成编译和上传新代码的全部工作。

上传开始后，可以看到 Arduino 电路板上的 Tx 和 Rx LED 灯闪烁，指示数据传输正在进行中。上传完成后，在消息区和控制台窗口显示上传操作完成的消息。

期间如果发生任何错误（串口选错、开发板型号选错等），在消息区和控制台窗口都会显示错误信息，如图 3.3。

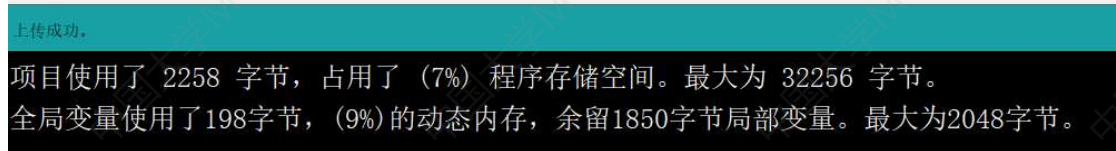
```
delay(1000);  
}
```



```
avrdude: error: programmer did not respond to command: exit bootloader  
avrdude: error: programmer did not respond to command: exit bootloader
```

图 3.3 上传失败信息

```
delay(1000);  
}
```



```
上传成功。  
项目使用了 2258 字节，占用了 (7%) 程序存储空间。最大为 32256 字节。  
全局变量使用了 198 字节，(9%) 的动态内存，余留 1850 字节局部变量。最大为 2048 字节。
```

图 3.4 上传成功信息

如果一切顺利，那么就可以在 Arduino 上运行程序了。

3.2.4 运行程序

一旦 Arduino IDE 显示上传过程完成，标注为 L 和 TX 的 LED 灯就开始闪烁。这表明程序正在运行。当上传过程完成后，bootloader 将自动重启 Arduino 并运行新程序。

digitalWrite 函数先将 13 引脚置 0（输出电压为 0 V），一秒钟后置 1（输出电压值为 5 V），从而使 L 灯闪烁。TX 灯闪烁是因为 Serial.print() 函数在向串口输出数据。

可以在 Arduino 中通过串口监视器观察串口输出内容。方法是，选择菜单栏中的 Tools>Serial Monitor 选项，或者单击工具栏上的串口监视器图标（放大镜图标）。串口监视器窗口中将显示从 Arduino 电路板上接收到的输出信息，如图 3.5 所示。

注意到，打开串口监视器后，blink 计数器的值重新从 1 开始计数。当打开串口监视器后，它会向 Arduino 发出复位信号。Arduino 上的 bootloader 重新加载应用程序，并从头开始运行。

也可以通过 Arduino 电路板上的复位按钮手动重启程序。Arduino Uno R3 上的复位按钮位于电路板的左上角。按下按钮然后释放，就可以复位 Arduino。

运行 Arduino 并不一定要求将其连接到计算机上。可以使用外部电源为其供电，比如电池盒，或者直流稳压电源，只要电源插头插入电路板上的插座即可。Arduino Uno R3 会自动检测是 USB 接口供电还是外部电源直接供电，然后启动程序运行。

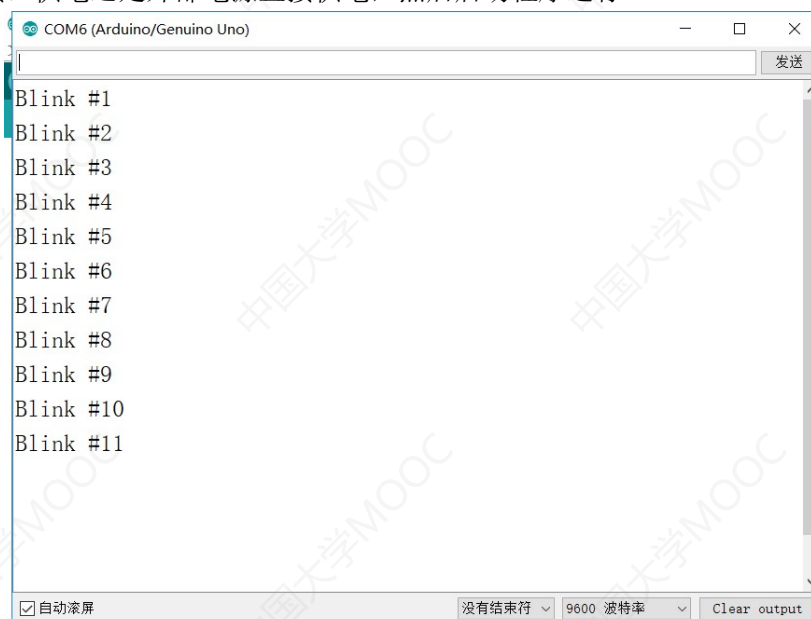


图 3.5 串口信息

3.3 连接硬件电路

虽然能够将程序上传到 Arduino 并运行起来是非常重要的一步，但是实际项目中需要完成的工作一定不止是让 LED 灯闪烁，真正要做的要比这多得多。这就是为什么项目中要接入某些外部电路的原因。本节讲述添加外部电路的基本知识。

3.3.1 使用扩展插槽

Arduino 的主要用途是使用输入/输出信号控制外部电路。为了做到这一点，需要通过 Arduino 的模拟信号或者数字信号连接到外部电路。这就是扩展插槽的作用。

Arduino 上的扩展插槽由位于电路板的顶部和底部的两排插槽组成（一些更高级的，Arduino 电路板，比如 Arduino Mega，还有第三个插槽，位于电路板右侧，以支持更多的接口），那么使用时将外部电路连接到插槽，就可以访问 Arduino 的输入/输出信号，并获得供电。



图 3.6 Arduino Uno 的上下两排插槽

表 3.1 Arduino Uno 下排插槽端口列表

标识	描述
IOREF	为微控制器提供参考电压（不是 5V 的情况下）
RESET	拉低时对 Arduino 复位
3.3V	为外部低电压电路提供的 3.3V 电压
5V	为外部电路提供的标准 5V 电压
GND	外部电路的接地线
GND	外部电路的另一个接地线
Vin	外部电路可以通过该端口为 Arduino 提供 5V 输入电压，替代 USB 和电源接口
A0	第 1 个模拟输入接口
A1	第 2 个模拟输入接口
A2	第 3 个模拟输入接口
A3	第 4 个模拟输入接口
A4	第 5 个模拟输入接口，也作为 TW1 接口的 SDA 引脚
A5	第 6 个模拟输入接口，也作为 TW1 接口的 SCL 引脚

表 3.2 Arduino Uno 上排插槽端口列表

标识	描述
AREF	供模拟输入端口使用的另一个参考电压（默认为 5 V）
GND	接地信号
13	数字端口 13，也作为 SPI 接口的 SCK 引脚
12	数字端口 12，也作为 SPI 接口的 MISO 引脚
~11	数字端口 11，PWM 输出端口，SPI 接口的 MOSI 引脚
~10	数字端口 10，PWM 输出端口，SPI 接口的 SS 引脚
~9	数字端口 9，PWM 输出端口
8	数字端口 8
7	数字端口 7
~6	数字端口 6，PWM 输出端口
~5	数字端口 5，PWM 输出端口
4	数字端口 4
~3	数字端口 3，PWM 输出端口
2	数字端口 2
TX	数字端口 1，串口输出引脚
RT	数字端口 0，串口输入引脚

3.3.2 使用面包板

连接这些微控制器主板还需要用到电路。在生成这些电路中起到重要的作用的就是面包板。使用面包板，完全不需要焊接就能制作出电路。

面包板表面排列有无数个小孔，把元件插进去就能制作出电路。如图 3.7 所示，这些小孔内部都是相连的，就像线一样。把元件跟跳线（导线）组合使用，马上就能组装好一个电

路。两侧的小孔是用来连接电源的，中央的凹槽设置正好能用来插入 IC 芯片。

原型设计初期基本上都会用到面包板。先在面包板上装一次电路，充分验证电路能正常运行后，再往电路板上安装。

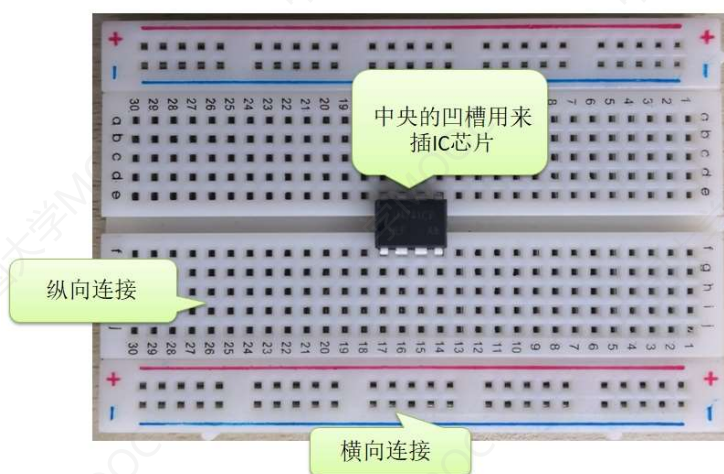


图 3.7 面包板的使用方法

3.3.3 使用原型扩展板

面包板是一个非常方便的工具，但如果想知道在设备上实现功能时，电路尺寸会有多大，使用起来感觉如何，就需要实际制作一个电路板。

制作电路板的一种方法是，把元件焊在一种叫作通用电路板的万能板上。连接元件时采用镀锡线和聚氯乙烯线来代替之前的跳线，这个方法在电路板少的时候是非常好用的，但是当要做的电路板增多时，也相应地就越费时间。

Arduino 原型扩展板是万能板的改进，按 Arduino 的尺寸加工，很方便地插在 Arduino Uno 板上。



图 3.8 原型扩展板连接 Arduino Uno 板

3.3.4 使用印刷电路板

制作电路板的另一种方法是使用印刷电路板。印刷电路板的表面上有小孔和连接这些小孔的线路图案，把元件插进小孔里再用焊接的方法固定，就能制成一个电路。如果需要重复制作很多相同的电路板，那么用这个方法制作会更简单。

大家可以按照以下步骤制作印刷电路板。

- ① 设计电路板的布线方式
- ② 按照设计来加工电路板

生成布线图首先需要专用的 CAD 软件。Altium Designer 软件是一款简单易用、原生 3D 设计增强的一体化设计环境。它结合了原理图、ECAD 库、规则和限制条件等特性的 PCB 设计工具。

对新手而言，困难的部分主要在于第二步，即加工电路板。印刷电路板的制作方法有雕刻法和蚀刻法（溶解电路板的表面）。两种方法都需要备齐好几种工具，还需要使用部分化学药品。最简单的方法，那就是把这一段外包。事实上有很多制造商只要接到步骤①里生成的 CAD 数据，就能制成印刷电路板。特别是需要制成较多数量的电路板时，不妨考虑这种方法。

3.4 小结

本章带领你完成了第一个 Arduino 项目。首先，在 Arduino IDE 编辑窗口输入程序代码，然后编译，最后将编译后的代码上传到 Arduino 上。同时也看到了如何使用串口监视器来观察程序输出。最后，介绍了如何搭建外部电路，以及如何连接到 Arduino。

第 4 章 实验项目

4.1 函数信号发生器的设计、组装与调试研究

一、实验目的

- 1、学习和掌握函数信号发生器的基本原理；
- 2、掌握 Arduino 编程原理；
- 3、学习和掌握 PCB 电路板的设计和制作；
- 4、掌握波形参数的测试方法。

二、实验原理和电路设计

在开始介绍项目之前，先说明什么是函数信号发生器。

函数发生器是产生某种波形的电路，主要有四种波形：方波、三角波、锯齿波和正弦波。

一个专用的函数信号发生器将花费上千元以上，当然要比我们在这里构建的系统功能要强大得多，我们这里只是给大家去进行一些有趣实验尝试，掌握其基本原理。

1、方波

采用 Arduino 产生简单的波形为方波，方波是高低逻辑电平之间简单循环。

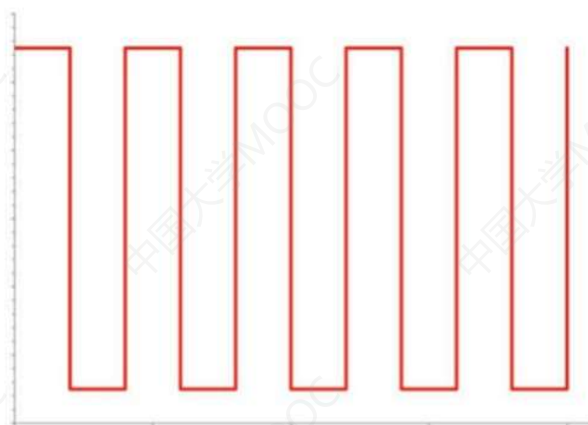


图 4.1 方波示意图

我们用来产生方波的电路几乎是可想象的最平常的电路。

事实上，我们需要做的只是连接 Arduino 的一个数字输出引脚（这里用 8 脚）和地。代码也很简单：

```
void setup()
{
    pinMode(8, OUTPUT);
}
void loop()
{
    // A total delay of 4 ms (frequency 250Hz)
    digitalWrite(8, HIGH);
    delay(2);
    digitalWrite(8, LOW);
}
```

```
delay(2);  
}
```

我们还可以使用 `delaymicroseconds()` 产生较小的延迟值。

事实上，Arduino 具有比这更容易产生一个方波的函数。例如，产生我们之前 250Hz 方波，我们可以使用：`tone(8, 250)` 代替 `digitalWrite` 语句。

让我们从在示波器上，看看这个方波到底是什么样子：



图 4.2 方波波形（占空比 50%）

除了波的频率，我们可能要修改一对其他主要特征参数：振幅（相对低电平，高电平有多高）和波的对称性，即所谓的占空比。

占空比只是测量高电平时间占总周期时间的比例。若方波是对称的：高低电平两种状态下都有相同的时间，即占空比为 50%。

如果我们要改变占空比，我们不能用 `tone()` 功能的设计。我们需要使用 `digitalwrite()` 和 `delay()` 功能。例如，产生一个占空比为 25% 的 250Hz 方波，在高电平后延迟 1 毫秒，然后低电平后延迟后 3ms。

使用示波器，我们就能看到波形的形状。

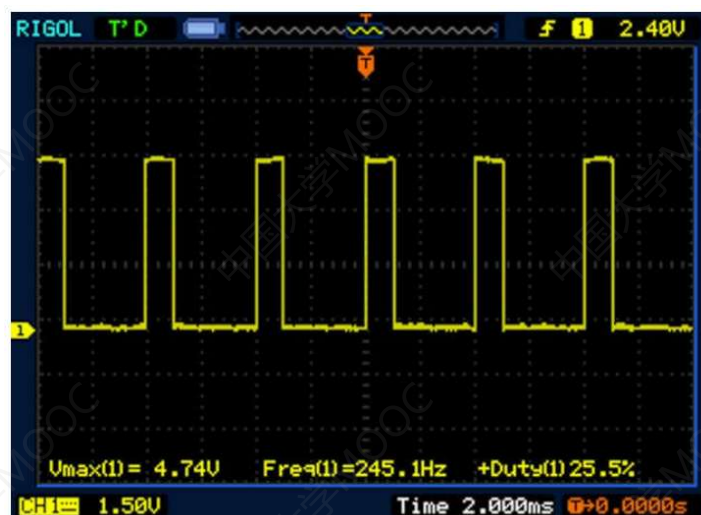


图 4.3 方波波形（占空比 25%）

时间坐标每格 2ms，我们可以清楚地看到脉冲宽度 1ms，脉冲之间的间隙为 3ms。如万用表测量电压，一个 50% 占空比的 250Hz 信号，平均值为 2.4V（正是 4.8 V 的一半，Arduino +5V 输出，测得的是 4.8V）。25% 占空比我们得到 1.2V（四分之一高电平）。

2、三角波

如何产生一个三角形波形，可以采用某种数字模拟转换器（或 DAC）。

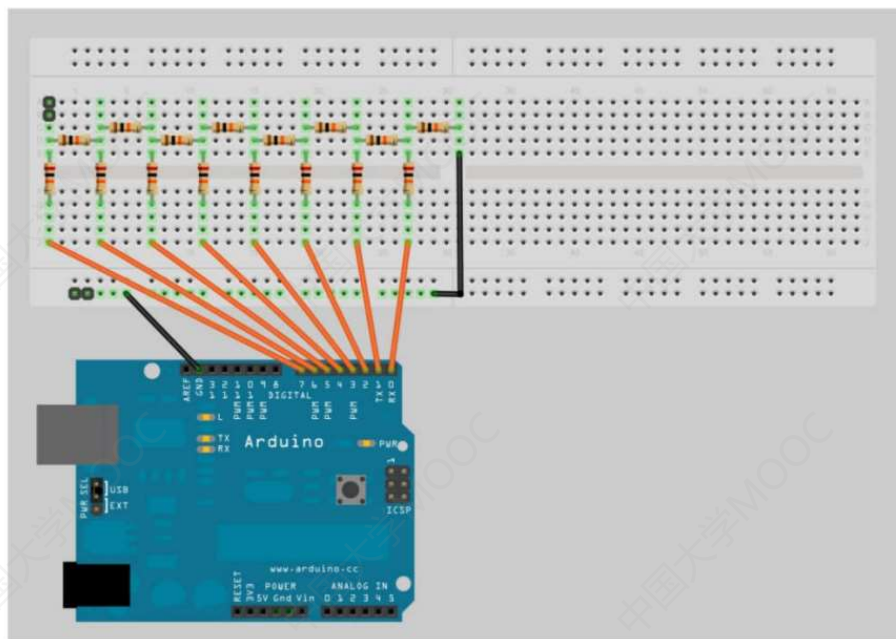


图 4.4 R-2R 电阻梯形网络

这是一个 8 位 DAC 电路称为 R-2R 电阻梯形网络。八位中的每一位对输出电压都有贡献。如果 8 位都为高电平，则输出约等于参考电压。如果最高位为低电平，就得到了大约参考电压一半。更确切地说，一个 8 位 DAC 如果 8 位都为高平，则得到 $(255 / 256) * \text{参考电压}$ ，最高位为低电平，我们得到 $(127 / 256) * \text{参考电压}$ 。一般地，DAC 输出为 $(x/256) * \text{参考电压}$ ，x 为 0 和 255 之间的值，。

产生三角波形程序为：

```
void setup()
{
    pinMode (0, OUTPUT);
    pinMode (1, OUTPUT);
    pinMode (2, OUTPUT);
    pinMode (3, OUTPUT);
    pinMode (4, OUTPUT);
    pinMode (5, OUTPUT);
    pinMode (6, OUTPUT);
    pinMode (7, OUTPUT);
}

void loop()
{
    for (int i=0; i<255; i++)
    {
        PORTD=i ;
    }
}
```



```

for(int i=255; i>0; i--)
{
  PORTD=i;
}
}

```

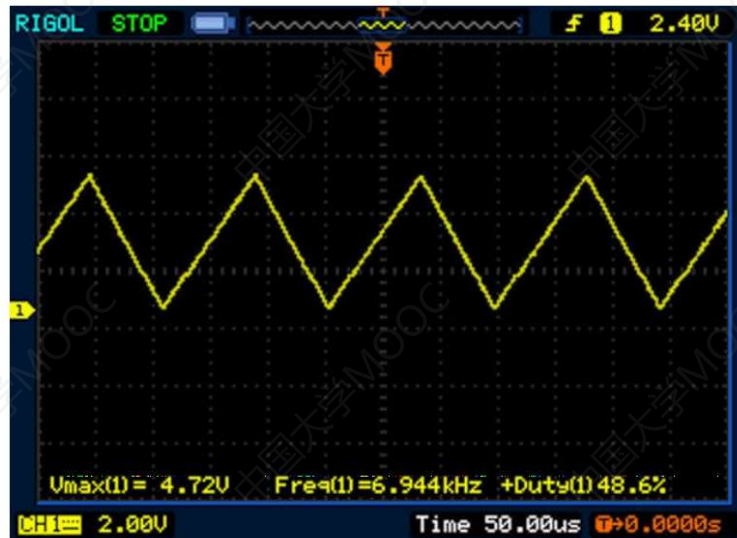


图 4.5 三角波波形

注意，这里我们使用 PORTD 代替单个输出引脚同时设置状态。这更快，并且确保所有的引脚在同一时间改变，这对产生平滑的模拟电压十分重要。

PORTD 是寄存器 D 端口，在这里其作用，是因为它映射到 0-7 数字引脚（8 位）；在一个操作，我们控制所有 8 引脚将二进制或十进制值的写入寄存器。例如，如果我们赋十进制值 123（这相当于 B01111011），将置 6, 5, 4, 3, 1, 0 为高电平。

现在的问题是，引脚 0、1 是用于通信的串行数据，所以利用 PORTD 意味着运行时我们不能使用任何串口通讯代码。然而，它是同时操作 8 个输出引脚的唯一方法（这种应用时，我们不再需要串行通信）。

我们可以非常容易地修改我们的代码，通过简单地注释（或者删除）loop 中的第二个 for 语句来产生锯齿波形。

3、正弦波

对于正弦波。我们需要预先计算随着时间推移的输出电压值（Arduino 不足以快到实时获取它们）。

正弦波一个周期为 2π 弧度——所以为了产生一个每个周期 256 时间步长的正弦波，对每个 x 值，计算 $y = \sin((x / 255) * 2\pi)$ 。因为正弦函数在 1 和 -1 间取值，而我们想要 0 和 255 之间的值，最简单的方法就是将浮点值乘以 128——给出一个 -128 到 128 间的值，然后加上 128 从而数值落在正确的范围内。

在 setup() 阶段，在全局数组存储结果。那么在 loop 中，我们需要做的是通过一次数组循环。程序代码如下：

```

#define PI 3.14
int sine[255];
void setup()

```

```

{
    pinMode(0, OUTPUT);
    pinMode(1, OUTPUT);
    pinMode(2, OUTPUT);
    pinMode(3, OUTPUT);
    pinMode(4, OUTPUT);
    pinMode(5, OUTPUT);
    pinMode(6, OUTPUT);
    pinMode(7, OUTPUT);
    float x;
    float y;
    for(int i=0; i<255; i++)
    {
        x = (float) i;
        y=sin((x / 255)*2*PI);
        sine[i]=int (y*128)+128;
    }
}
void loop()
{
    for(int i=0; i<255; i++)
    {
        PORTD=sine[ i ];
        delayMicroseconds(10);
    }
}

```

如果我们运行这个代码，并用 DAC(如 R-2R 电阻梯形网络)，在示波器上我们就会发现波形确实非常像正弦波。

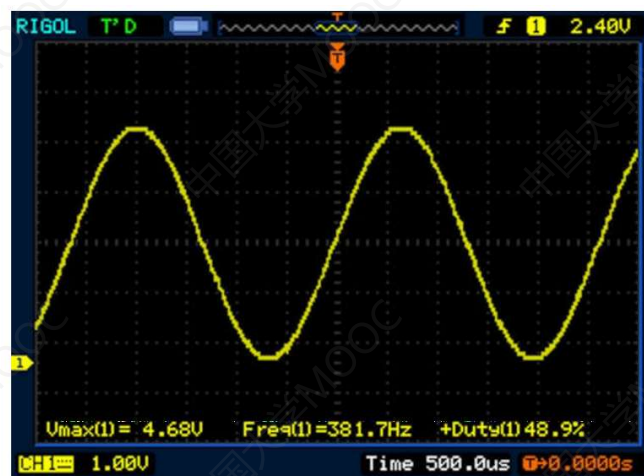


图 4.6 正弦波波形

三、实验任务与要求

- 1、用 Arduino UNO 设计一个函数信号发生器，要求如下：

- (1) 输出可在方波、三角波、正弦波间切换。
 - (2) 输出频率自定。
 - (3) 输出幅度 0.5~2V 可调 (要求加输出缓冲)。
- 2、设计的电路, 调整元件参数, 直至满足任务要求。重点考虑:
- (1) 如何切换波形 (硬件、软件方式);
 - (2) DAC 方法;
 - (3) 单电源运放缓冲设计。
- 3、绘制程序流程图, 编制与调试函数信号发生器程序。
- 4、制作 Arduino 扩展板, 并将其与 UNO 板组装联调。

四、思考题

- 1、如何改变方波的占空比?
- 2、试分析简单的函数信号发生器中影响波形频率的因素。
- 3、列举几个 DAC 方法, 并简要说明原理。

4.2 电容测量仪的设计、制作与调试

一、实验目的

- 1、学习和掌握电容测量仪的基本原理；
- 2、掌握 Arduino 编程原理；
- 3、学习和掌握 PCB 电路板的设计和制作；
- 4、学习 LCD1602 液晶显示屏的使用。

二、实验原理

电容的测量是基于对 RC 电路的时间常数的计算，由脉冲电路原理可知，电容的充电速度与 R 和 C 的大小有关，R 与 C 的乘积越大，过渡时间就越长。这个 RC 的乘积就叫做 RC 电路的时间常数 τ ，即 $\tau = R \cdot C$ 。若 R 的单位用欧姆，C 的单位用法拉，则 τ 的单位为秒。

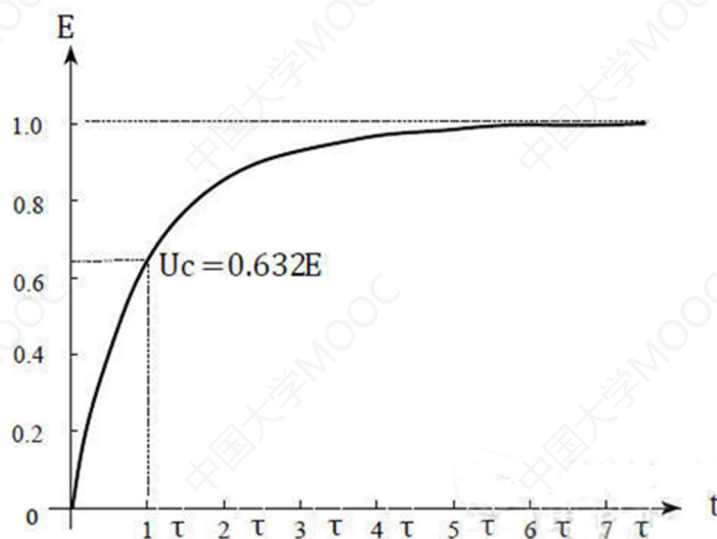


图 4.7 电容测量原理

图示曲线可以得到充电过程的一般规律： U_c 是按指数规律上升的， U_c 开始变化较快，以后逐渐减慢，并缓慢地趋近其最终值，当 $t = \tau$ 时， $U_c = 0.632E$ ；本测量仪就是利用 Arduino 测量 $U_c = 0$ 到 $0.632E$ 这段时间，用下列式子计算被测电容值：

$$C = \frac{\tau}{R} \quad (4-1)$$

三、实验设计方案

电容测量仪由 RC 电路、测量显示部分和 Arduino UNO 板组成，本项目硬件工作主要是设计制作扩展板，它由 RC 电路和显示电路组成，并将其插在 UNO 板上。

RC 电路比较简单，由电阻 R 和待测电容 C 组成。电阻 R 的选择，可根据时间常数 τ 来确定。可通过切换电阻，实现多量程。待测电容需要插拔，因而要设计合适的插座。

测量值的显示，可采用 LCD 显示模块。LCD 显示模块有一个优势就是将驱动电路集成在模块中。LCD 显示模块具有标准，不同的生产商所生产的大多数模块都可以按相同方法来使用。1602 液晶在应用中非常广泛，最初的 1602 液晶使用的是 HD44780 控制器，现在各个厂家的 1602 模块基本上都是采用了与之兼容的 IC，所以特性上基本都是一致的。有关 LCD1602 模块的内容，详见附录 1。

四、软件设计

我们给出电容测试程序，供大家参考。

程序中首先导入 LCD1602 库，硬件按 4 数据口模式连线。

```
#include <LiquidCrystal.h> //声明 1602 液晶的函数库
```

```
LiquidCrystal lcd(12,11,5,4,3,2); //4 数据口模式连线声明
```

导入库后，定义了 3 个使用的引脚的常量。引脚 A1 被用作模拟输入，测量电容两端的电压。待测电容根据电容值大小，选择通过 $1\text{ k}\Omega$ 电阻 R_1 或 $1\text{ M}\Omega$ 电阻 R_2 充电。这两个电阻通过引脚 A2 和 A3 控制。如果 A2 或 A3 引脚被设定为输入，则与之相连的电阻不参与电容的充电或放电过程。如果它被设定为输出并置低电位，电容将通过它放电。如果它设为输出和高电位，则电容将通过它充电。

```
const int stepHighCPin=A2;
```

```
const int stepLowCPin=A3;
```

```
const int sensePin=A1;
```

常量也用于定义两个电阻的值。

```
const float R1=1000.0;
```

```
const float R2=1000000.0;
```

threshold 常量是在记录用 R_1 或 R_2 给电容充电所需时间的时候，被认为是已经充分充电的模拟读数值(0 到 1023)。这就是时间常数，是 5 V 满挡读数的 63%。

```
const int threshold=645; //63%
```

本项目电容测量仪的基本原理是，测量电容器充电达到一定的水平需要的时间。这要求模拟测量要尽可能快。

下面的代码行，先使测量从每秒约 9000 个读数加快到每秒 58000 个读数。

```
//speed up ADC
```

```
const byte PS_128=(1<<ADPS2) | (1<<ADPS1) | (1<<ADPS0);
```

```
const byte PS_16=(1<<ADPS2);
```

```
void setup()
```

```
{
```

```
  ADCSRA &= ~PS_128; // 清除分频系数 128
```

```
  ADCSRA |= PS_16; // 设置分频系数为 16
```

```
  pinMode(stepHighCPin, INPUT); //设置电阻  $R_1$  的控制引脚为输入
```

```
  pinMode(stepLowCPin, INPUT); //设置电阻  $R_2$  的控制引脚为输入
```

```
  lcd.begin(16,2); //初始化 LCD 显示器
```

```
  lcd.clear();
```

```
  lcd.print("Capacitance");//在顶行显示信息 Capacitance
```

```
}
```

loop 函数首先需要使用 readC 函数读取高电容值的电容。如果这个读数太低了，它快速地切换量程，灵敏度提高 1000 倍，再读取另一个读数。然后调用 display 在 LCD 上显示读数，等待半秒之后再次启动。

```
void loop()
```

```
{
```

```
  float highRange = readC(R1, stepHighCPin, stepLowCPin);
```

```
  float reading=highRange;
```

```
  if(highRange<1.0F)
```

```
  {
```

```

    float lowRange=readC(R2, stepLowCPin, stepHighCPin);
    reading=lowRange;
  }
  display(reading);
  delay(500);
}

```

该 readC 函数首先调用 discharge 放空电容器的电荷，然后测量通过指定的引脚(stepPin)需要多长时间来给电容充电。电容的微法值可以通过所用时间的微秒数除以所用电阻的欧姆值来计算。

```

float readC(float r, int stepPin, int unusedPin)
{
    discharge();
    long t0=micros();
    charge(stepPin, unusedPin);
    long t1=micros();
    float T=float(t1-t0);
    float C=T / r;
    return C;
}

```

我们希望放电要尽可能快，所以放电总是使用 $1k\Omega$ 电阻 R_1 进行。放电函数启用 R_1 ，然后等待直到电容的模拟读数达到 0。

```

void discharge()
{
    pinMode(stepLowCPin, INPUT);
    pinMode(stepHighCPin, OUTPUT);
    digitalWrite(stepHighCPin, LOW);
    while(analogRead(sensePin)>0){};
}

```

当在电容器充电测试时，被使用的电阻引脚设置为输出和 HIGH。未使用的电阻控制引脚设置为输入。charge 函数主要是等待循环，直到读数达到阈值。

```

void charge(int stepPin, int unusedPin)
{
    pinMode(unusedPin, INPUT);
    pinMode(stepPin, OUTPUT);
    digitalWrite (stepPin, HIGH);
    while(analogRead(sensePin) < threshold) {};
}

```

在 LCD 上显示内容有 3 种可能途径，由 display 函数控制。如果读数低于 $0.001(1nF)$ ，显示屏显示 Too Low。如果它大于 $0.8\mu F$ ，那么它将以微法显示读数；否则以纳法显示。

```

void display(float reading)
{

```



```

if(reading<0.001)
{
    clearRow();
    lcd.print(" Too Low ");
}
else if(reading>0.8)
{
    clearRow();
    lcd.print(reading);
    lcd.print(" uF" );
}
else
{
    clearRow();
    lcd.print(reading*1000.0);
    lcd.print("nF");
}
}

clearRow 函数在显示新的值之前用来清空显示器的底行。
void clearRow()
{
    lcd.setCursor(0,1);
    lcd.print( " " );
    lcd.setCursor(0,1);
}

```

五、实验任务与要求

- 1、用 Arduino UNO 设计电容测量仪，要求如下：该仪表可以测量大于 1nF 到几百 μ F 的电容值。
- 2、设计电路，画出电路原理图，调整元件参数，直至满足任务要求。
- 3、编制与调试电容测量仪程序。
- 4、制作扩展板，并将其与 UNO 板组装联调。

4.3 温度检测显示控制器的设计与制作

一、实验目的

- 1、学习和掌握温度检测显示控制器的基本原理；
- 2、掌握 Arduino 编程原理；
- 3、学习和掌握 PCB 电路板的设计和制作；
- 4、学习旋转编码器和 LCD1602 液晶显示屏的使用。

二、实验设计方案

本项目使用 Arduino UNO 扩展板进行温度检测显示控制。硬件框图如图 4.8 所示，下面对各部分模块进行说明。

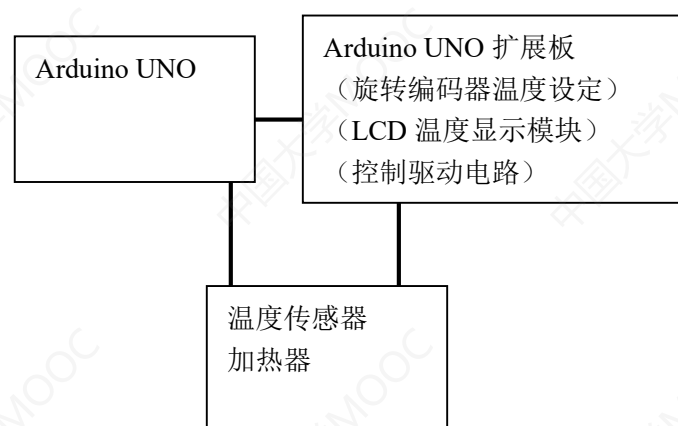


图 4.8 温度检测显示控制器框图

LCD 显示模块

本次实验使用 Arduino UNO 直接驱动 1602 液晶显示字母和数字，LCD 显示模块有一个优势就是将驱动电路集成在模块中。LCD 显示模块具有标准，不同的生产商所生产的大多数模块都可以按相同方法来使用。1602 液晶在应用中非常广泛，最初的 1602 液晶使用的是 HD44780 控制器，现在各个厂家的 1602 模块基本上都是采用了与之兼容的 IC，所以特性上基本都是一致的。有关 LCD1602 模块的内容，详见附录 1。

旋转编码器

旋转编码器是一种数字器件，这种器件有两个输出（A 和 B）。当旋转旋钮时，输出会有变化，这个变化就告诉你旋钮是顺时针旋转还是逆时针旋转。有关旋转编码器的内容，详见附录 2。

温度传感器

LM35 是美国国家半导体公司生产的，具有很高的工作精度和较宽的线性工作范围，工作电压范围为 4~30V。其精度为 0.5°C ，输出电压与摄氏温度成线性比例关系，非线性温度误差低于 $\pm 0.25^{\circ}\text{C}$ ，线性温度系数为 $+10\text{mV}/^{\circ}\text{C}$ ，无需外部校准或微调。输出电压与温度的表达式为： $U_{out} = 10\text{mV}/^{\circ}\text{C} \times t^{\circ}\text{C}$ 。



图 4.9 LM35 的供电模式

LM35 的供电模式有单电源与正负双电源两种模式。正负双电源供电模式可提供 -55°C ~ 150°C 的全量程范围测量；单电源供电模式在 25°C 下电流约为 $50\mu\text{A}$ ，温度测量范围 0°C ~ 100°C ，其输出电压范围为 $0\sim 1\text{V}$ 。

控制驱动电路

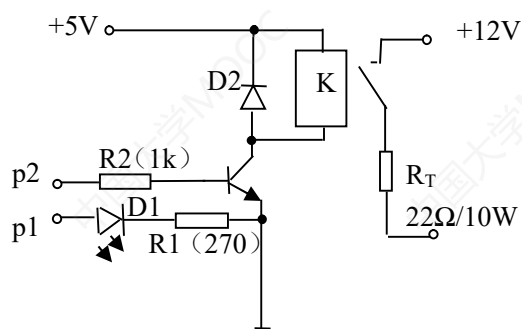


图 4.10 控制驱动电路

图 4.10 中，p1、p2 为 Arduino UNO 接口，可自选。二极管 D1 用于指示继电器工作状态，二极管 D2 的作用是防止继电器断开时产生的反向电动势的影响。

三、硬件设计

设计参考电路图如下：

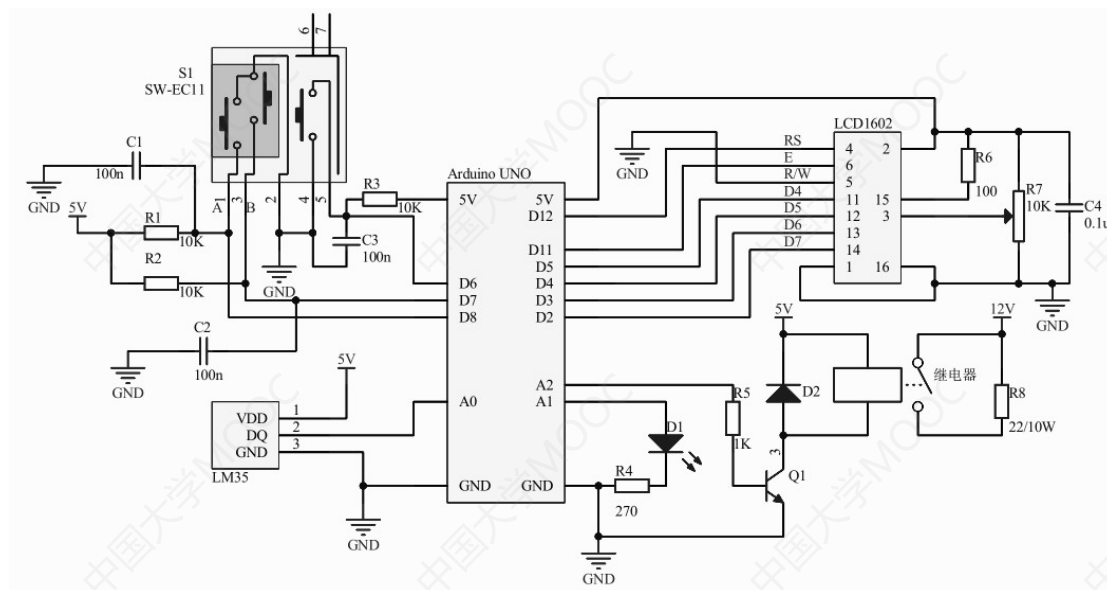


图 4.11 电路原理图

四、软件设计

设计参考程序:

```
#include <LiquidCrystal.h>
//LiquidCrystal display with:
//rs on pin 12
//enable on pin 11
//d4-7 on pins 5-2
LiquidCrystal lcd(12,11,5,4,3,2);

int ledPin = 15;
int relayPin = 16;
int aPin = 8;
int bPin = 7;
int buttonPin = 6;
int analogPin = 0;

float setTemp = 20.0;
float measureTemp;
char mode = 'C'; //can be changed to F
boolean override = false;
float hysteresis = 0.25;

void setup() {
    lcd.begin(2,20);
    pinMode(ledPin,OUTPUT);
    pinMode(relayPin,OUTPUT);
    pinMode(aPin,INPUT);
    pinMode(bPin,INPUT);
    pinMode(buttonPin,INPUT);
    lcd.clear();
}

void loop() {
    static int count = 0;
    measureTemp = readTemp();
    if(!digitalRead(buttonPin))
    {
        override = !override;
        updateDisplay();
        delay(500);//debounce
    }
    int change = getEncoderTurn();
    setTemp = setTemp + change *0.1;
    if(count == 1000)
    {
```

```

        updateDisplay();
        updateOutputs();
        count = 0;
    }
    count++;
}

int getEncoderTurn()
{
    // return -1, 0, or +1
    static int oldA=LOW;
    static int oldB=LOW;
    int result = 0;
    int newA=digitalRead(aPin);
    int newB=digitalRead(bPin);
    if (newA != oldA || newB != oldB)
    {
        // something has changed
        if(oldA == LOW && newA==HIGH)
        {
            result = -(oldB*2-1);
        }
    }
    oldA = newA;
    oldB = newB;
    return result;
}

float readTemp()
{
    long a =analogRead(analogPin);
    float temp = 500.0*a/1023.0;
    return temp;
}

void updateOutputs()
{
    if(override||measureTemp<setTemp-hysteresis)
    {
        digitalWrite(ledPin,HIGH);
        digitalWrite(relayPin,HIGH);
    }
    else if(!override&&measureTemp>setTemp+hysteresis)
    {
        digitalWrite(ledPin,LOW);
    }
}

```

```

        digitalWrite(relayPin,LOW);
    }
}
void updateDisplay()
{
    lcd.setCursor(0,0);
    lcd.print("Actual:");
    lcd.print(adjustUnits(measureTemp));
    lcd.print("o");
    lcd.print(mode);
    lcd.print(" ");

    lcd.setCursor(0,1);
    if(override)
    {
        lcd.print("OVERRIDE ON");
    }
    else
    {
        lcd.print("Set:");
        lcd.print(adjustUnits(setTemp));
        lcd.print("o");
        lcd.print(mode);
        lcd.print(" ");
    }
}
float adjustUnits(float temp)
{
    if(mode == 'C')
    {
        return temp;
    }
    else
    {
        return(temp*9)/5+32;
    }
}

```

控制系统中的迟滞问题考虑

设计这样一个温度控制器时需考虑到要防止“摆动”情况的出现。一个简单的通断控制系统，就会出现摆动情况。当温度降至设定值之下时，电源接通，开始加热，直至温度高于设定值，电源断开。然后慢慢变冷，直至温度再次低于设定值，此时加热器又被接通，如此循环往复。但当温度恰好平衡在转变温度时，这种摆动会非常频繁，这种高频切换容易损坏器件。解决方法之一，就是引入迟滞（hysteresis）。图 4.12 展示如何利用一个迟滞值（迟滞

变量设为 0.25°C) 避免高频摆动的方法。

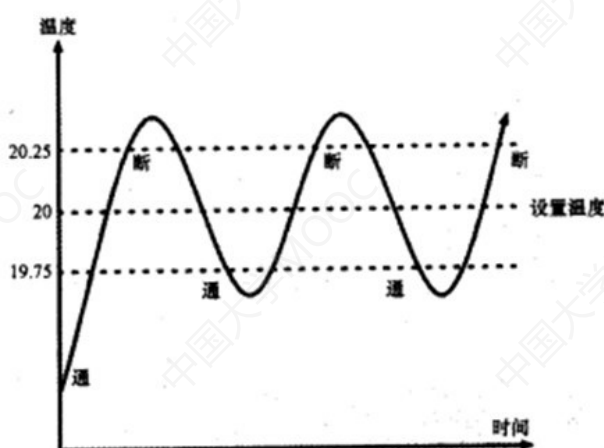


图 4.12 控制系统中的迟滞

温度随着电源接通而升高，它会到达设置点。然而，在温度超出设置点加上迟滞值之前电源并不会断开。同样，当温度下降时，降到低于设置点时并不会重新加电，而是只有当温度下降到低于设置点减去迟滞值时才会加电。

显示闪烁问题考虑

编程时不要连续刷新显示值，因为读数的任何细微变化都可能导致显示乱闪烁。因此不要在每次主循环中刷新显示，而是循环多次（如 1000 次）刷新一次。可引入一个变量（如 counter），该变量每个循环增加 1 次，当其增加到 1000 时，刷新显示并将 counter 变量复位到 0。

每次改变显示值都使用 `lcd.clear()` 也会导致其闪烁。因此，可以采用将新温度值写在旧温度值的上面。

五、实验任务与要求

1、用 Arduino UNO 设计温度检测显示控制器，要求如下：

（1）选用合适的温度检测传感器，设计一个能实现测量范围 $0^{\circ}\text{C}\sim 100^{\circ}\text{C}$ ，测量精度为 0.5°C 的温度检测转换电路；

（2）设计一个温度加热控制电路，实现用旋转编码器设定控制温度值，当检测温度值低于设定值时，加热电路工作；当检测温度值高于设定值时，加热电路停止加热。

2、设计的电路，完成相应器件的选择和参数计算，制作扩展板和控制加热电路。

3、编制与调试温度检测显示控制器程序。

4、将制作的扩展板、控制加热电路板与 Arduino UNO 板组装后，进行系统联调。

4.4 倒计时定时器的设计、制作与调试

一、实验目的

- 1、学习掌握用 Arduino UNO 设计倒计时定时器；
- 2、学习掌握 PCB 电路板的设计和制作；
- 3、学习掌握 Arduino UNO 扩展板的设计与制作；
- 4、学习掌握旋转编码器的使用。

二、实验设计方案

在日常生活中常常需要定时器,通过设定时间来通知我们一项任务完成或者下一项任务应该开始。本项目使用 Arduino UNO 与其扩展板来实现倒计时定时器。硬件框图如图 4.13 所示,下面对各部分模块进行说明。



图 4.13 倒计时定时器框图

1、旋转编码器

旋转编码器是一种数字器件,这种器件有两个输出(A 和 B)。当旋转旋钮时,输出会有变化,这个变化就告诉你旋钮是顺时针旋转还是逆时针旋转。有关旋转编码器的内容,详见附录 2。

2、LED 时间显示模块

(1) 7 段数码管结构

7 段数码管一般由 8 个发光二极管组成,其中由 7 个细长的发光二极管组成数字显示,另外一个圆形的发光二极管显示小数点。

当发光二极管导通时,相应的一个点或一个笔画发光。控制相应的二极管导通,就能显示出各种字符,尽管显示的字符形状有些失真,能显示的字符数量也有限,但其控制简单,使用也方便。发光二极管的阳极连在一起的称为共阳极数码管,阴极连在一起的称为共阴极数码管,如图 4.14 所示。

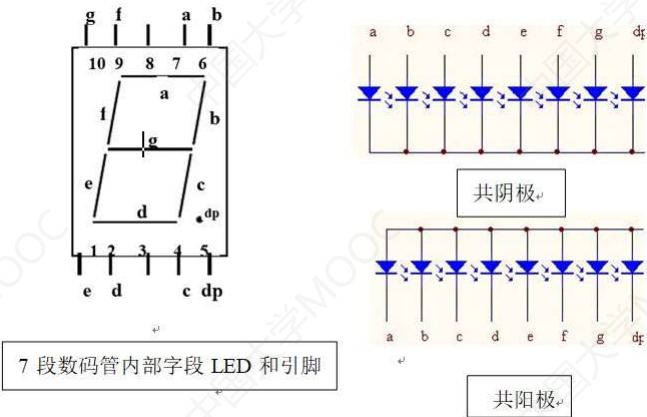


图 4.14 7 段数码管结构图

(2) 7 段数码管驱动方法

发光二极管(LED 是一种由磷化镓(GaP)等半导体材料制成的,能直接将电能转变成光能的发光显示器件。当其内部有一电流通过时,它就会发光。

7 段数码管每段的驱动电流和其他单个 LED 发光二极管一样,一般为 5~10mA;正向电压随发光材料不同表现为 1.8~2.5V 不等。

7 段数码管的显示方法可分为静态显示与动态显示,下面分别介绍。

1) 静态显示驱动:

静态驱动也称直流驱动。静态驱动是指每个数码管的每一个段码都由一个单片机的 I/O 脚进行驱动,或者使用如 BCD 码二-十进位计数器进行驱动。静态驱动的优点是编程简单,显示亮度高,缺点是占用 I/O 脚多,如驱动 5 个数码管静态显示则需要 $5 \times 8 = 40$ 根 I/O 脚来驱动。

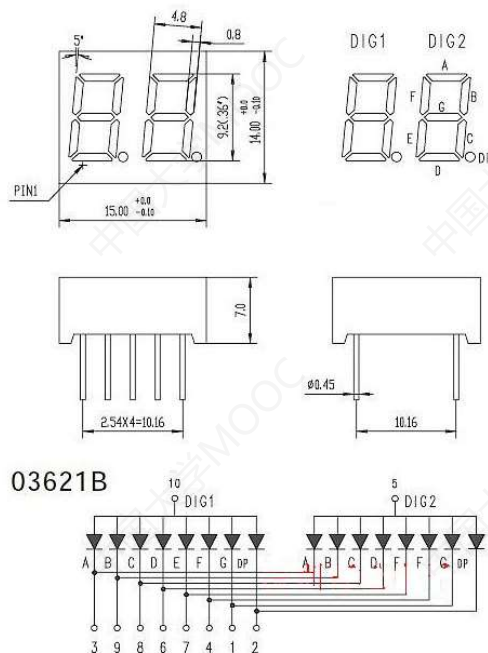
2) 动态显示驱动:

动态驱动是将所有数码管的 8 个显示笔划"a,b,c,d,e,f,g,dp"的同名端连在一起,另外为每个数码管的公共极 COM 增加位元选通控制电路,位元选通由各自独立的 I/O 线控制,当单片机输出字形码时,所有数码管都接收到相同的字形码,但究竟是那个数码管会显示出字形,取决于单片机对位元选通 COM 端电路的控制,所以我们只要将需要显示的数码管的选通控制打开,该位元就显示出字形,没有选通的数码管就不会亮。

透过分时轮流控制各个 LED 数码管的 COM 端,就使各个数码管轮流受控显示,这就是动态驱动。在轮流显示过程中,每位元数码管的点亮时间为 1~2ms,由于人的视觉暂留现象及发光二极管的余辉效应,尽管实际上各位数码管并非同时点亮,但只要扫描的速度足够快,给人的印象就是一组稳定的显示资料,不会有闪烁感,动态显示的效果和静态显示是一样的,能够节省大量的 I/O 埠,而且功耗更低。

由于时间显示分和秒,只需 4 个数码管,可考虑两个 2 位 7 段发光 LED,封装和引脚图如下图所示。

0.36 英寸 二位



3、时间报警电路

报警可采用蜂鸣器。蜂鸣器按工作原理可分为压电式及电磁式的二大类：压电式蜂鸣器主要由多谐振荡器、压电蜂鸣片、阻抗匹配器及共鸣箱、外壳等组成。它是以压电陶瓷的压电效应，来带动金属片的振动而发声；电磁式的蜂鸣器，则是用电磁的原理，通电时将金属振动膜吸下，不通电时依振动膜的弹力弹回。

蜂鸣器按是否含有驱动，又分为有源蜂鸣器和无源蜂鸣器两种。“有源”是指蜂鸣器本身内含驱动了，直接给它一定的电压就可以响；“无源”是需要靠外部的驱动才可以响的。注意，这里的“源”不是指电源。而是指振荡源。也就是说，有源蜂鸣器内部带振荡源，所以只要一通电就会叫。而无源内部不带振荡源，所以如果用直流信号无法令其鸣叫，要用 2k~5k 的方波去驱动它。无源蜂鸣器的优点是便宜、声音频率可控（可以做出“多来米发索拉西”的效果）以及在一些特例中，可以和 LED 复用同一个控制口。而有源蜂鸣器的优点在于程序控制方便。

4、设计思路

定时器总是处于两种状态之一：停止定时与定时运行。在停止定时状态，转动旋转编码器就可以改变定时时间；在定时运行状态，定时器倒计时。按下旋转编码器上的按钮可以对这两种状态进行切换。

转动旋转编码器时定时时间并不是以秒为步进长度来改变时间，我们有一个标准时间数组来实现。

EEPROM 库用于保存最后使用的时间，所以每当这个装置上电时，它将记住最后一次使用的时间。

5、硬件

设计参考电路图如图 4.15 所示。

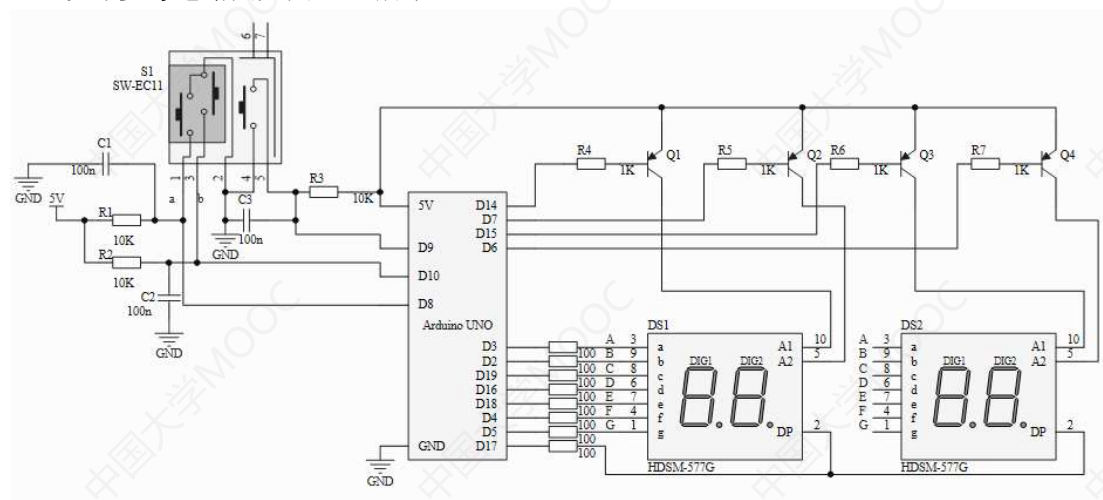


图 4.15 电路原理图

6、软件设计

参考程序为：

```
#include<EEPROM.h>
int segmentPins[] = {3,2,19,16,18,4,5,17};
int displayPins[] = {6,15,7,14};
int times[] = {5,10,15,20,30,45,100,130,200,230,300,400,500,600,700,800,900,1000, 1500, 2000, 3000};
int numTimes=19;
```

```

byte selectedTimeIndex;
int timerMinute;
int timerSecond;
int buzzerPin=11;
int aPin=8;
int bPin=10;
int buttonPin=9;

boolean stopped = true;
byte digits[10][8] = {
    //a  b  c  d  e  f  g  .
    {1,1,1,1,1,0,0},//0
    {0,1,1,0,0,0,0},//1
    {1,1,0,1,1,0,1},//2
    {1,1,1,1,0,0,1},//3
    {0,1,1,0,0,1,1},//4
    {1,0,1,1,0,1,1},//5
    {1,0,1,1,1,1,1},//6
    {1,1,1,0,0,0,0},//7
    {1,1,1,1,1,1,1},//8
    {1,1,1,1,0,1,1},//9
};

void setup()
{
    for(int i=0;i<8;i++)
    {
        pinMode(segmentPins[i],OUTPUT);
    }
    for(int i=0;i<4;i++)
    {
        pinMode(displayPins[i],OUTPUT);
    }
    pinMode(buzzerPin,OUTPUT);
    pinMode(buttonPin,INPUT);
    pinMode(aPin,INPUT);
    pinMode(bPin,INPUT);
    selectedTimeIndex=EEPROM.read(0);
    timerMinute=times[selectedTimeIndex]/100;
    timerSecond=times[selectedTimeIndex]%100;
}

void loop()
{

```

```

    if(!digitalRead(buttonPin))
    {
        stopped = !stopped;
        digitalWrite(buzzerPin,LOW);
        while(!digitalRead(buttonPin)){};
        EEPROM.write(0,selectedTimeIndex);
    }
    updateDisplay();
}
void updateDisplay() //mmss
{
    int minsecs = timerMinute*100+timerSecond;
    int v = minsecs;
    for(int i=0;i<4;i++)
    {
        int digit=v%10;
        setDigit(i);
        setSegments(digit);
        v=v/10;
        process();
    }
    setDigit(5);// all digits off to prevent uneven illumination
}

void process()
{
    for(int i=0;i<100; i++)//tweak this number between flicker and blur
    {
        int change=getEncoderTurn();
        if(stopped)
        {
            changeSetTime(change);
        }
        else
        {
            updateCountingTime();
        }
    }
    if(timerMinute == 0 && timerSecond == 0)
    {
        digitalWrite(buzzerPin,HIGH);
    }
}

```



```

void changeSetTime(int change)
{
    selectedTimeIndex+=change;
    if(selectedTimeIndex<0)
    {
        selectedTimeIndex=numTimes;
    }
    else if(selectedTimeIndex>numTimes)
    {
        selectedTimeIndex = 0;
    }
    timerMinute=times[selectedTimeIndex]/100;
    timerSecond=times[selectedTimeIndex]%100;
}

void updateCountingTime()
{
    static unsigned long lastMillis;
    unsigned long m=millis();
    if(m>(lastMillis+1000) && (timerSecond>0 || timerMinute>0))
    {
        digitalWrite(buzzerPin,HIGH);
        delay(10);
        digitalWrite(buzzerPin,LOW);
        if(timerSecond==0)
        {
            timerSecond=59;
            timerMinute--;
        }
        else
        {
            timerSecond--;
        }
        lastMillis = m;
    }
}

void setDigit(int digit)
{
    for(int i=0;i<4;i++)
    {
        digitalWrite(displayPins[i],(digit!=i));
    }
}

```

```

void setSegments(int n)
{
    for(int i=0;i<8; i++)
    {
        digitalWrite (segmentPins[i], !digits[n][i]);
    }
}

int getEncoderTurn()
{
    // return -1, 0, or +1
    static int oldA=LOW;
    static int oldB=LOW;
    int result = 0;
    int newA=digitalRead(aPin);
    int newB=digitalRead(bPin);
    if (newA != oldA || newB != oldB)
    {
        // something has changed
        if(oldA == LOW && newA==HIGH)
        {
            result = -(oldB*2-1);
        }
    }
    oldA = newA;
    oldB = newB;
    return result;
}

```

三、实验任务与要求

- 1、用 Arduino UNO 设计倒计时定时器，要求如下：设定倒计时时间若干（设定标准时间数组），通过旋转编码器选择，时间到时报警（上面例子中是数码管是共阳的，**要求按数码管是共阴的情况设计**）。
- 2、设计电路，完成相应器件的选择，制作 Arduino UNO 扩展板。
- 3、编制与调试倒计时定时器程序。
- 4、将制作的扩展板与 Arduino UNO 板组装后，进行系统联调。

四、思考题

- 1、考虑倒计时定时器的应用，例如如何控制定时加热，简要介绍实现的电路。
- 2、如何通过修改程序，消除秒钟滴答声。

4.5 Arduino 实时音频处理实验

一、实验目的

- 1、学习掌握用 Arduino UNO 进行实时音频信号处理；
- 2、学习掌握 PCB 电路板的设计和制作；
- 3、学习掌握 Arduino UNO 音频处理扩展板的设计与制作。

二、实验设计方案

数字信号处理，并不是数字信号处理器（DSP）独占的领域。Arduino 具有进行实时音频信号的处理的能力。

1、可调正弦波信号源

Arduino 能产生的不仅数字信号，而且是高质量的波形。输出信号是连续的正弦波，可在较宽的频率范围内调整。压控振荡器（VCO）是电子合成器中最重要的元件之一。它被用来创建基本的音调，并由滤波器，混频器和包络变成了电子合成声。

可通过调节电压来改变信号源的频率。使用一个 10k 的电位器，使 UNO 板的 A0 的输入电压改变，从而改变输出信号的频率。得到的正弦波发生器，可以在许多音频领域中应用。

在下面示例程序中，包含两个函数。

第一，void calcsine()，计算一个正弦波表。一系列正弦值不是外部预计算的，而是由单片机本身的一次计算得到。

第二，ISR (TIMER2_OVF_vect) 函数每 16 μ s 被调用，以频率 62.5kHz 自动发生中断。中断函数第四次调用时，获取的 A0 模拟输入值被用于 loop() 中正弦波频率的设置。

在 setup()，将 Timer2 设置为快速 PWM 模式，而中断是在上述时间间隔触发。这些通过直接访问 ATmega328 的定时器寄存器来设置。

压控振荡器程序示例如下：

```
// Voltage-Controlled Oscillator
#define sbi(PORT,bit) (PORT |=1<<bit)
#define cbi(PORT,bit) (PORT &=~(1<<bit))
const int pwmOutPin = 11; //PWM output
const int testPin = 3;
const float pi = 3.141592;
int indexCnt, interCnt; //index, interrupt counters
byte sine[512]; //array for sine values
// interrupt control variable
volatile boolean adcSample;
volatile byte bufferAdc0;

void setup () {
  pinMode(pwmOutPin, OUTPUT); // sets the PWM pin as output
  pinMode(testPin, OUTPUT); // sets the test pin as output
  calcSine(); // load memory with sine table
  // set adc prescaler to 64, 8-Bit ADC and VCC Reference
  // select analog channel 0
  ADCSRA |= 0b00000110;
  ADMUX |= 0b01100000;
```

```

// set Timer2 to fast PWM mode
TCCR2A |= 0b10000011;
// Timer2 no prescaler
TCCR2B |= 0b00000001;
cbi(TCCR2B, CS21);
cbi(TCCR2B, CS22);
// disable Timer0, enable Timer2 interrupt
cbi(TIMSK0, TOIE0);
sbi(TIMSK2, TOIE2);

void loop () {
    while (!adcSample) {} // wait for new ADC samples
    digitalWrite (testPin, HIGH);
    adcSample = false;
    OCR2A = sine(indexCnt); // send current value to PWM
    indexCnt++;
    indexCnt += bufferAdc0;
    indexCnt &= 511; // constraint counter index to between 0 and 511
    digitalWrite (testPin, LOW);
}

// calculate sine array with 512 values between 0 and 2 * pi
void calcSine () {
    for (int n = 0; n <= 511; n++) {
        sine [n] = 127 * sin (pi * n / 255) + 128;
    }
}

// get potentiometer value
ISR (TIMER2_OVF_vect) {
    interCnt++; // reduce sampling rate 62.5 kHz through 15.625 kHz
    if (4 == interCnt) {
        bufferAdc0 = ADCH; // get 8-bit ADC value
        adcSample = true;
        sbi (ADCSRA, ADSC); // start next conversion
        interCnt = 0;
    }
}

```

2、数字信号处理

除了产生高质量的模拟信号，Arduino 也可以作为一个信号处理器。在这一部分中，我们以混响发生器为例来说明。

以模拟技术产生人工混响一直赋有挑战性。高、低和带通滤波器的制作相对简单，但产生混响是困难的。先是采用诸如混响板或弹簧的机械装置进行尝试，后来，采用组桶式存储

器 (bucket brigade storage)。

在微控制器中，混响效果可以使用循环缓冲存储器实现。为此，模拟信号首先像往常一样通过 ADC 数字化。Arduino 的 ADC 工作在输入电压 0 和 5V 之间，所以一个 2.5 V 的偏置电压必须被添加到输入信号上。通过耦合电容，将调整后在 0 至 5V 的范围内的信号发送到 ADC 输入端。图 4.16 为用于此目的的简单电路。

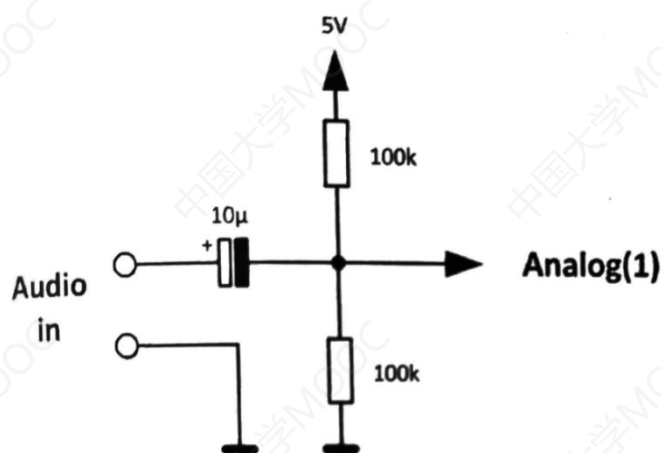


图 4.16 混响发生器输入电路

数字化值存储在数组中。为了产生的混响效果所需的延迟，反复读取循环缓冲区。延时用一单独的电位器调节。

由于 ATmega328 的资源有限，我们限制数组为 512 字节。

在环形缓冲区，这个存储器阵列构成一个循环中，一个指针用于访问写入数据的内存位置和第二个指针来访问读取位置。完成一个步骤，两个指针都指向下一个位置。如果指针到达数组的结尾（511），指针又被设置为 0，旧的数据将被覆盖。读写指针之间的距离决定了时间延迟的长度。

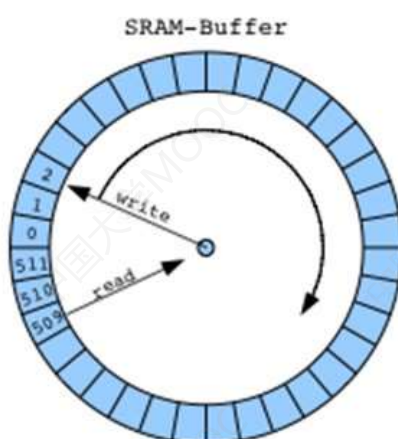


图 4.17 环形缓冲区

在微控制器中，混响效果可以使用循环缓冲存储器实现。数字化值存储在数组中。为了产生的混响效果所需的延迟，反复读取循环缓冲区。延时用一单独的电位器调节。

与前面在正弦波发生器相同，使用快速 PWM 模式，在 `setup()` 中的相关的设置也相同。

这里，电位器值和模拟输入信号都由中断程序采样。为此，交替地读出 A1（模拟输入）

和 A0（电位器）。

在主程序中，循环缓冲器被写入并在延迟后读取，时间延迟与 A0 处的电位器电压成比例。

混响发生器示例程序

```
// Reverb Generator
#define cbi (PORT,bit) (PORT &=~(1 << bit))
#define sbi (PORT,bit) (PORT |=1 << bit)
const int pwmOutPin = 11; // PWM output
const float pi = 3.1415926535;
const byte halfMax = 127;
boolean div2, selChan; // control variables for sampling
// interrupt variables accessed globally
volatile boolean adcSample;
volatile byte potSampler, analogSample;
int indexCounter, valAnalog, valPot;
byte pwmBuf, ringbuff[512]; // Audio Ringbuffer Array 8-Bit

void setup () {
    pinMode (pwmOutPin, OUTPUT); // set the PWM pin as output
    // adjust adc prescaler
    cbi (ADCSRA, ADPS2); sbi (ADCSRA, ADPS1); sbi (ADCSRA, ADPS0);
    sbi (ADMUX, ADLAR); sbi (ADMUX, REFS0); cbi (ADMUX, REFS1);

    // Set muxer ADC0
    cbi {ADMUX, MUX0}; cbi (ADMUX, MUX1); cbi (ADMUX, MUX2); cbi (ADMUX,
    MUX3);

    // fast PWM for Timer2
    cbi (TCCR2A, COM2A0); sbi (TCCR2A, COM2A1);
    sbi (TCCR2A, WGM20); sbi (TCCR2A, WGM21); cbi (TCCR2B, WGM22);
    // set Timer2 Prescaler
    sbi (TCCR2B, CS20); cbi(TCCR2B, CS21); cbi (TCCR2B, CS22);
    // Timer2 PWM Port Enable
    sbi ( DDRB, 3); // set digital pin 11 to output
    //cli (); // disable interrupts to avoid distortion
    cbi (TIMSK0, TOIE0); // disable Timer0 !!! delay is off now
    sbi (TIMSK2, TOIE2); // enable Timer2 Interrupt
    valAnalog = analogSample;

    void loop () {
        while (! adcSample) {}
        adcSample = false;
        // scale delayed sample with pot value
        valPot = (halfMax - ringbuff[indexCounter]) * potSample/255;
```

```

valAnalog = halfMax - analogSample;
valAnalog = valAnalog + valPot;

// limit analog value (min/max)
if (valAnalog < -halfMax) valAnalog = -halfMax;
if (valAnalog > halfMax) valAnalog = halfMax;

// add offset to PWM buffer
pwmBuf = halfMax + valAnalog;
ringbuff[indexCounter] = pwmBuf;
indexCounter++;

// reduce buffer index to 0...511 intervall
indexCounter = indexCounter & 511;

OCR2A = pwmBuf; // write current sample to PWM channel

// get analog and pot value
ISR (TIMER2_OVF_vect) {
    PORTB = PORTB | 1;
    div2 = ! div2; // reduce sampling frequency by factor 2
    if (div2) {
        selChan = ! selChan;
        if (selChan) {
            potSample = ADCH;
            sbi (ADMUX, MUX0); // set muxer to ADC1
        }
        else {
            analogSample = ADCH;
            cbi (ADMUX, MUX0); // set muxer to ADC0
            adcSample = true;
        }
        sbi (ADCSRA, ADSC); // start next conversion
    }
}

```

3、音频输出放大器

使用扬声器可以获得较好的音频质量。问题是，其中一个阻抗仅仅几欧姆，因此不能直接连接到 `aduino` 板的引脚，为此需要一个放大器，`LM386` 一个非常合适我们应用的放大器，它只需要几个额外元件构成电路，如图 3 所示。除了放大器，还具有音频滤波器的功能。`2.2uF` 电容器负责放大器的直流去耦，并且具有一定的高通特性，而 `1 μF` 电容器提供低通滤波。当使用简单的脉宽调制产生音频信号时，高频 `PWM` 谐波将被很好地抑制，而在较低的千赫范围内所需的音频频率传递到放大器。

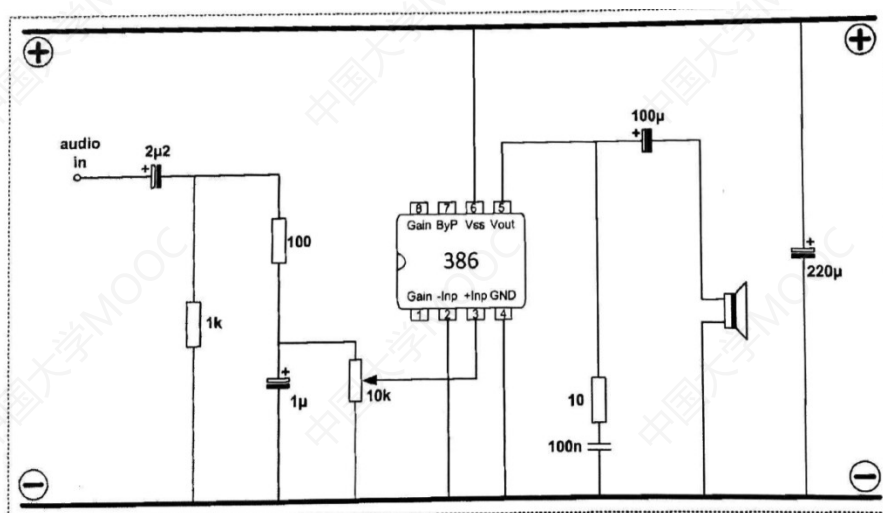


图 4.18 基于 LM386 音频放大器电路图

三、实验任务与要求

- 1、用 Arduino UNO 实现音频信号处理设计，在 1) 音频正弦波信号源 2) 混响发生器中选择一个电路，并分别加上 3) 音频输出放大器，形成一个完整的电路，完成相应器件的选择，制作 Arduino UNO 扩展板。
- 3、编制与调试相关程序。
- 4、将制作的扩展板与 Arduino UNO 板组装后，进行系统联调。

4.6 多功能数字时钟的设计与制作

一、实验目的

- 1、学习掌握用 Arduino UNO 设计数字时钟；
- 2、学习掌握 PCB 电路板的设计和制作；
- 3、学习掌握 Arduino UNO 扩展板的设计与制作；
- 4、学习掌握 DS1302 时钟芯片和 LCD1602 液晶显示屏的使用。

二、实验设计方案

本项目使用 Arduino UNO 设计多功能数字时钟。硬件框图如图 1 所示，下面对各部分模块进行说明。

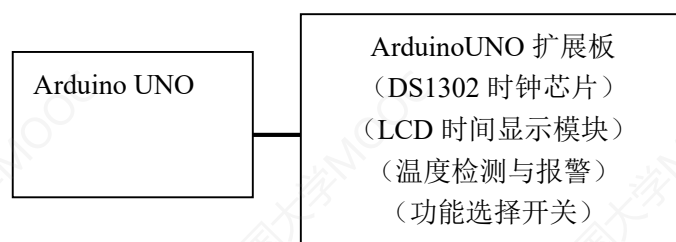


图 4.19 多功能数字时钟框图

DS1302 时钟芯片

DS1302 是美国 DALLAS 公司推出的一种高性能、低功耗的实时时钟芯片，附加 31 字节静态 RAM，采用 SPI 三线接口与 CPU 进行同步通信，并可采用突发方式一次传送多个字节的时钟信号和 RAM 数据。实时时钟可提供秒、分、时、日、星期、月和年，一个月小与 31 天时可以自动调整，且具有闰年补偿功能。工作电压宽达 2.5~5.5V。采用双电源供电（主电源和备用电源），可设置备用电源充电方式，提供了对后背电源进行涓细电流充电的能力。详细内容，可查阅芯片的 Datasheet。

LCD 显示模块

本次实验使用 Arduino UNO 直接驱动 1602 液晶显示字母和数字，LCD 显示模块有一个优势就是将驱动电路集成在模块中。LCD 显示模块具有标准，不同的生产商所生产的大多数模块都可以按相同方法来使用。1602 液晶在应用中非常广泛，最初的 1602 液晶使用的是 HD44780 控制器，现在各个厂家的 1602 模块基本上都是采用了与之兼容的 IC，所以特性上基本都是一致的。有关 LCD1602 模块的内容，详见附录 1。

温度传感器

温度传感器可采用 LM35，具体内容参见“4.3 温度检测显示控制器的设计与制作”。

三、硬件设计

设计参考电路图如下：

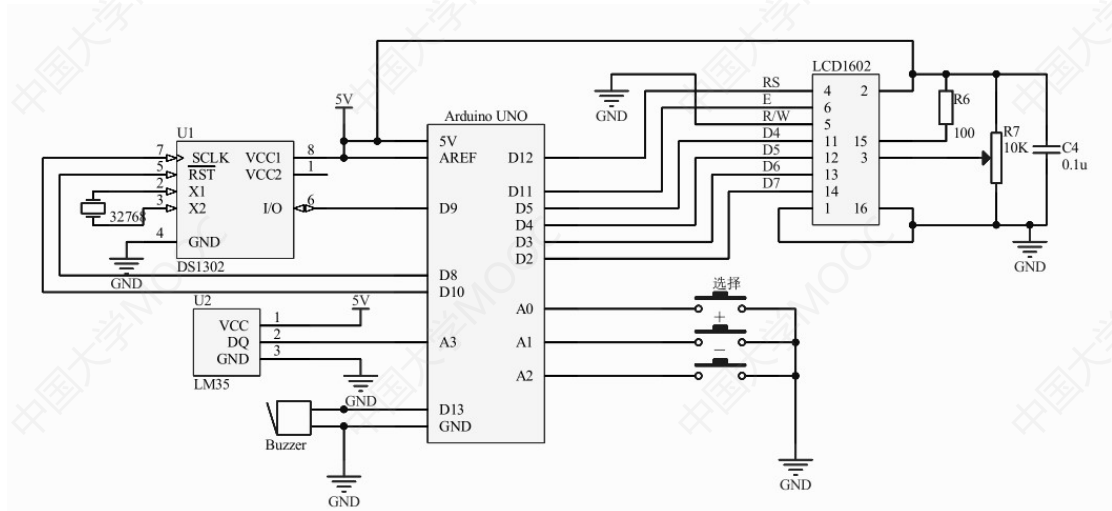


图 4.20 电路原理图

四、软件设计

设计参考程序如下：

```
/*
 * LCD RS pin to digital pin 12
 * LCD Enable pin to digital pin 11
 * LCD D4 pin to digital pin 5
 * LCD D5 pin to digital pin 4
 * LCD D6 pin to digital pin 3
 * LCD D7 pin to digital pin 2
 * LCD R/W pin to ground
 * LCD VSS pin to ground
 * LCD VCC pin to 5V
 */
#include <DS1302.h>
#include <LiquidCrystal.h> //LCD1602 显示头文件

LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

#define choose A0 //选择端口
#define add A1 //加
#define minus A2 //减
#define Tone 13 //蜂鸣器端口
uint8_t CE_PIN = 8; //DS1302 RST 端口
uint8_t IO_PIN = 9; //DS1302 DAT 端口
uint8_t SCLK_PIN = 10; //DS1302 CLK 端口
DS1302 rtc(CE_PIN, IO_PIN, SCLK_PIN); //创建 DS1302 对象
unsigned long seconds;
int s = 0, m = 0, h = 0, d = 0, mon = 0, y = 0; //时间进位
int second = 0, minute = 0, hour = 0, day = 0, month = 0, year = 0; //当前时间
```

```

int SECOND = 0, MINUTE = 0, HOUR = 0, DAY = 0, MONTH = 0, YEAR = 0; //初始时间
int chose = 0, alarm_choose = 0, ButtonDelay = 10, frequency = 2093;
int alarm_hour = 7, alarm_minute = 30, alarm_second = 0; //闹钟时间
float Temperatures, Temp_Alarm = 30 ;

```

```

/** 格式化输出 */

```

```

void FormatDisplay(int col, int row, int num){
    lcd.setCursor(col, row);
    if(num < 10)    lcd.print("0");
    lcd.print(num);
}

```

```

/** 计算时间 */

```

```

void time() {
    second = (SECOND + seconds) % 60;    //计算秒
    m = (SECOND + seconds) / 60;        //分钟进位
    FormatDisplay(6,1,second);

    minute = (MINUTE + m) % 60;    //计算分钟
    h = (MINUTE + m) / 60;        //小时进位
    FormatDisplay(3,1,minute);

    hour = (HOUR + h) % 24;    //计算小时
    d = (HOUR + h) / 24;        //天数进位
    FormatDisplay(0,1,hour);

    lcd.setCursor(2, 1);
    lcd.print(":");
    lcd.setCursor(5, 1);
    lcd.print(":");
}

```

```

/** 根据年月计算当月天数 */

```

```

int Days(int year, int month){
    int days = 0;
    if (month != 2){
        switch(month){
            case 1: case 3: case 5: case 7: case 8: case 10: case 12: days = 31;    break;
            case 4: case 6: case 9: case 11:    days = 30;    break;
        }
    }else{    //闰年
        if(year % 4 == 0 && year % 100 != 0 || year % 400 == 0){
            days = 29;
        }
    }
}

```

```

        else{
            days = 28;
        }
    }
    return days;
}

/** 计算当月天数 */
void Day(){
    int days = Days(year,month);
    int days_up;
    if(month == 1){
        days_up = Days(year - 1, 12);
    }
    else{
        days_up = Days(year, month - 1);
    }
    day = (DAY + d) % days;
    if(day == 0){
        day = days;
    }
    if((DAY + d) == days + 1 ){
        DAY -= days;
        mon++;
    }
    if((DAY + d) == 0){
        DAY += days_up;
        mon--;
    }
    FormatDisplay(8,0,day);
}

/** 计算月份 */
void Month(){
    month = (MONTH + mon) % 12;
    if(month == 0){
        month = 12;
    }
    y = (MONTH + mon - 1) / 12;
    FormatDisplay(5,0,month);
    lcd.setCursor(7, 0);
    lcd.print('-');
}

```

```
/** 计算年份 */
```

```
void Year(){
    year = ( YEAR + y ) % 9999;
    if(year == 0){
        year = 9999;
    }
    lcd.setCursor(0, 0);
    if(year < 1000){
        lcd.print("0");
    }
    if(year < 100){
        lcd.print("0");
    }
    if(year < 10){
        lcd.print("0");
    }
    lcd.print(year);
    lcd.setCursor(4, 0);
    lcd.print('-');
}
```

```
/** 根据年月日计算星期几 */
```

```
void Week(int y,int m, int d){
    if(m == 1){
        m = 13;
    }
    if(m == 2){
        m = 14;
    }
    int week = (d+2*m+3*(m+1)/5+y+y/4-y/100+y/400)%7+1;
    String weekstr = "";
    switch(week){
        case 1: weekstr = "Mon. "; break;
        case 2: weekstr = "Tues. "; break;
        case 3: weekstr = "Wed. "; break;
        case 4: weekstr = "Thur. "; break;
        case 5: weekstr = "Fri. "; break;
        case 6: weekstr = "Sat. "; break;
        case 7: weekstr = "Sun. "; break;
    }
    lcd.setCursor(11, 0);
    lcd.print(weekstr);
}
```

```
/** 显示时间、日期、星期 */
```

```
void Display() {  
    time();  
    Day();  
    Month();  
    Year();  
    Week(year,month,day);  
}
```

```
/** 显示光标 */
```

```
void DisplayCursor(int rol, int row) {  
    lcd.setCursor(rol, row);  
    lcd.cursor();  
    delay(100);  
    lcd.noCursor();  
    delay(100);  
}
```

```
/** 设置初始时间 */
```

```
void set(int y, int mon, int d, int h, int m, int s){  
    YEAR = y;  
    MONTH = mon;  
    DAY = d;  
    HOUR = h;  
    MINUTE = m;  
    SECOND = s;  
}
```

```
/** 通过按键设置时间 */
```

```
void Set_Time(int rol, int row, int &Time){  
    DisplayCursor(rol, row);  
    if(digitalRead(add) == LOW){  
        delay(ButtonDelay);  
        if(digitalRead(add) == LOW){  
            Time ++;  
        }  
        Display();  
    }  
    if(digitalRead(minus) == LOW){  
        delay(ButtonDelay);  
        if(digitalRead(minus) == LOW){  
            Time --;  
        }  
        Display();  
    }  
}
```



```

}

/** 按键选择 */
void Set_Clock(){
    if(digitalRead(choose)==LOW){
        lcd.setCursor(9, 1);
        lcd.print("SetTime");
        while(1){
            if(digitalRead(choose) == LOW){
                delay(ButtonDelay);
                if(digitalRead(choose) ==LOW){
                    chose++;
                }
            }
            seconds = millis()/1000;
            Display();
            if(chose == 1){
                Set_Time(1, 1, HOUR);          //SetHour
            }else if(chose == 2){
                Set_Time(4, 1, MINUTE);        //SetMinute
            }else if(chose == 3){
                Set_Time(7, 1, SECOND);        //SetSecond
            }else if(chose == 4){
                Set_Time(9, 0, DAY);           //SetDay
            }else if(chose == 5){
                Set_Time(6, 0, MONTH);         // SetMonth
            }else if(chose == 6){
                Set_Time(3, 0, YEAR);          //SetYear
            }else if(chose >= 7) {
                chose = 0;
                break;
            }
        }
    }
}

```

```

/** 设置闹钟小时 */
void Set_Alarm_Hour(){
    DisplayCursor(1, 1);
    if(digitalRead(add) == LOW){
        delay(ButtonDelay);
        if(digitalRead(add) == LOW){
            alarm_hour ++;
            if(alarm_hour == 24){

```

```

        alarm_hour = 0;
    }
    FormatDisplay(0,1,alarm_hour);
}
}
if(digitalRead(minus) == LOW){
    delay(ButtonDelay);
    if(digitalRead(minus) == LOW){
        alarm_hour--;
        if(alarm_hour == -1){
            alarm_hour = 23;
        }
        FormatDisplay(0,1,alarm_hour);
    }
}
}

/** 设置闹钟分钟 */
void Set_Alarm_Minute(){
    DisplayCursor(4, 1);
    if(digitalRead(add) == LOW) {
        delay(ButtonDelay);
        if(digitalRead(add) == LOW){
            alarm_minute++;
            if(alarm_minute == 60){
                alarm_minute = 0;
            }
            FormatDisplay(3,1,alarm_minute);
        }
    }
    if(digitalRead(minus) == LOW){
        delay(ButtonDelay);
        if(digitalRead(minus) == LOW){
            alarm_minute--;
            if(alarm_minute == -1){
                alarm_minute = 59;
            }
            FormatDisplay(3,1,alarm_minute);
        }
    }
}

/** 设置报警温度 */
void Set_Alarm_Temp(){

```

```

DisplayCursor(10, 1);
if(digitalRead(add) == LOW) {
    delay(ButtonDelay);
    if(digitalRead(add) == LOW){
        Temp_Alarm ++;
    }
}
if(digitalRead(minus) == LOW){
    delay(ButtonDelay);
    if(digitalRead(minus) == LOW){
        Temp_Alarm --;
    }
}
}
}

```

/** 进入报警设置 */

```

void Set_Alarm(){
    if(digitalRead(add) == LOW && digitalRead(minus) == LOW){
        alarm_hour = hour;
        alarm_minute = minute;
        //alarm_choose = 1;
        lcd.setCursor(0, 0);
        lcd.print("set alarm      ");
        lcd.setCursor(6, 1);
        lcd.print("00");          //闹钟秒数
        while(1){
            if(digitalRead(choose) == LOW){
                delay(ButtonDelay);
                if(digitalRead(choose) == LOW){
                    alarm_choose++;
                }
            }
            lcd.setCursor(9, 1);
            lcd.print(Temp_Alarm);
            lcd.setCursor(14, 1);
            lcd.print((char)223);    //显示 o 符号
            lcd.setCursor(15, 1);
            lcd.print("C");          //显示字母 C
            if(alarm_choose == 1){
                Set_Alarm_Hour();
            }else if(alarm_choose == 2){
                Set_Alarm_Minute();
            }else if(alarm_choose == 3){

```

```

        Set_Alarm_Temp();
    }else if(alarm_choose >= 4){
        alarm_choose = 0;
        break;
    }
}
}
}

/** 正点蜂鸣 */
void Point_Time_Alarm(){
    if(minute == 0 && second == 0){
        tone(Tone,frequence);
        delay(500);
        noTone(Tone);
    }
}

/** 闹钟 指定时间蜂鸣 */
void Clock_Alarm(){
    if(hour == alarm_hour && minute == alarm_minute && second == alarm_second){
        tone(Tone,frequence);
        delay(5000);
        noTone(Tone);
    }
}

/** 获取 LM35 温度 */
void GetTemperatures(){
    long a = analogRead(A3); // Get temperatures
    Temperatures = 500.0*a/1023.0;
    lcd.setCursor(9, 1);
    lcd.print(Temperatures); //获取温度
    lcd.setCursor(14, 1);
    lcd.print((char)223); //显示 o 符号
    lcd.setCursor(15, 1);
    lcd.print("C"); //显示字母 C
}

/** 超过指定温度报警 */
void Temperatures_Alarm(){
    if(Temperatures >= Temp_Alarm){
        tone(Tone,frequence);
    }
}

```

```

        delay(500);
        noTone(Tone);
    }
}

void setup(){
    for(int i = 2; i <= 13; i++){
        pinMode(i, OUTPUT);
    }
    digitalWrite(add, HIGH);
    digitalWrite(minus, HIGH);
    digitalWrite(choose, HIGH);
    lcd.begin(16, 2); //初始化 LCD1602
    Time t;
    t.hour=12;t.min=59;t.sec=59;
    t.year=2020;t.mon=5;t.date=7;
    rtc.setTime(t.hour,t.min,t.sec); //设置 DS1302 芯片初始时间
    rtc.setDate(t.year,t.mon,t.date);
    set(t.year,t.mon,t.date,t.hour,t.min,t.sec);
    // set(rtc.year(), rtc.month(), rtc.date(), rtc.hour(), rtc.minutes(), rtc.seconds()); //通电读取
    DS1302 芯片的时间
    rtc.writeProtect(false); // 关闭 DS1302 芯片写保护
    rtc.halt(false); //为 true 时 DS1302 暂停
}

void loop() {
    seconds = millis()/1000; //获取单片机当前运行时间
    Display(); //显示时间
    Set_Clock(); //设置时间
    Set_Alarm(); //设置闹钟
    Point_Time_Alarm(); //正点蜂鸣
    Clock_Alarm(); //闹钟时间蜂鸣
    GetTemperatures(); //获取 LM35 温度
    Temperatures_Alarm(); //超过指定温度报警

    //断电将单片机的当前时间写到 DS1302 芯片中
    Time t;
    t=rtc.getTime();
    rtc.setTime(t.hour,t.min,t.sec); //设置 DS1302 芯片初始时间
    rtc.setDate(t.year,t.mon,t.date);
}

```

五、实验任务与要求

- 1、用 Arduino UNO 设计多功能数字时钟，要求实现功能：
 - (1) 显示时间、日期和星期
 - (2) 断电保存时间
 - (3) 通过按钮设置时间、日期
 - (4) 整点响铃
 - (5) 自定义闹钟
 - (6) 显示温度
 - (7) 自定义报警温度
 - (8) 按键功能：按选择键进入设置时间功能；同时按 + - 键进入闹钟和报警温度设置功能；
 - (9) 再按选择键光标跳动，光标跳到哪，当前的参数即可通过加减键修改。
- 2、设计的电路，完成相应器件的选择和参数计算，制作 Arduino UNO 扩展板。
- 3、编制与调试多功能数字时钟程序。
- 4、将制作的 Arduino UNO 扩展板与 Arduino UNO 板组装后，进行系统联调。

4.7 Arduino 万用表的设计与制作

一、实验目的

- 1、学习掌握用 Arduino UNO 设计万用表；
- 2、学习掌握 PCB 电路板的设计和制作；
- 3、学习掌握 Arduino UNO 扩展板的设计与制作；
- 4、学习掌握 LCD1602 液晶显示屏的使用。

二、实验设计方案

本项目使用 Arduino UNO 设计万用表。硬件框图如图 4.21 所示，下面对各部分模块进行说明。

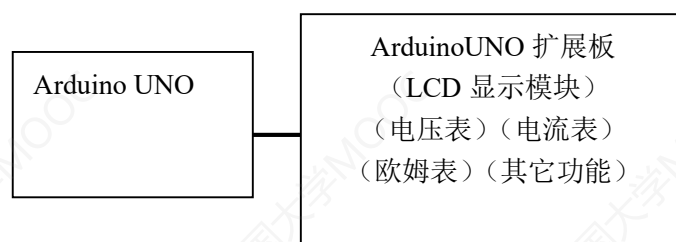


图 4.21 Arduino 万用表框图

LCD 显示模块

本次实验使用 Arduino UNO 直接驱动 1602 液晶显示字母和数字，LCD 显示模块有一个优势就是将驱动电路集成在模块中。LCD 显示模块具有标准，不同的生产商所生产的大多数模块都可以按相同方法来使用。1602 液晶在应用中非常广泛，最初的 1602 液晶使用的是 HD44780 控制器，现在各个厂家的 1602 模块基本上都是采用与之兼容的 IC，所以特性上基本都是一致的。有关 LCD1602 模块的内容，详见附录 1。

三、硬件设计

万用表由电压表、电流表、欧姆表及其它功能模块组成，下面就各个部分进行说明：

- 1、电压表的简化图。三个量程可以通过 Arduino 扩展板加按钮进行选择。在进行电压测量时，只有一个开关是闭合的。

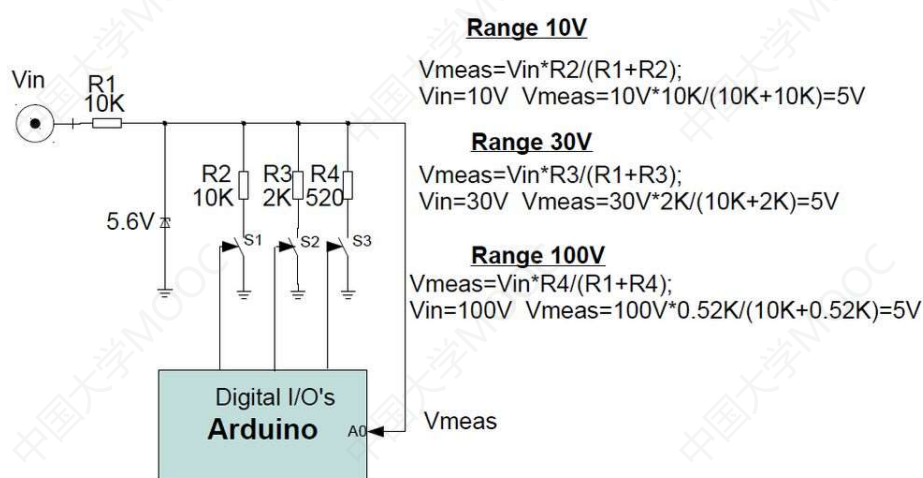


图 4.22 电压表电路简图

2、电流表的简化图。被测量电流流过 1Ω 电阻到地，其输出经放大后连接到 Arduino 的 A1 接口，放大器的增益为 10。为了进行过流保护，配置了一个 500mA 的复位 PTC。

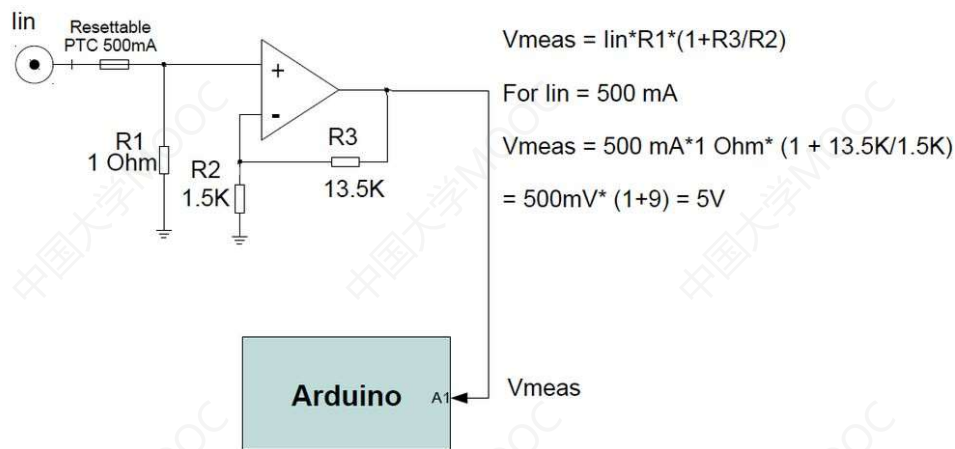


图 4.23 电流表电路简图

3、欧姆表的简化图。由齐纳二极管相对于正电压源产生基准电压，该电压被施加到由晶体管和运算放大器组成的电流转换器上，作为电压输入，晶体管的射极接可变电阻。此时齐纳二极管上的电压和该电阻上的电压相等。

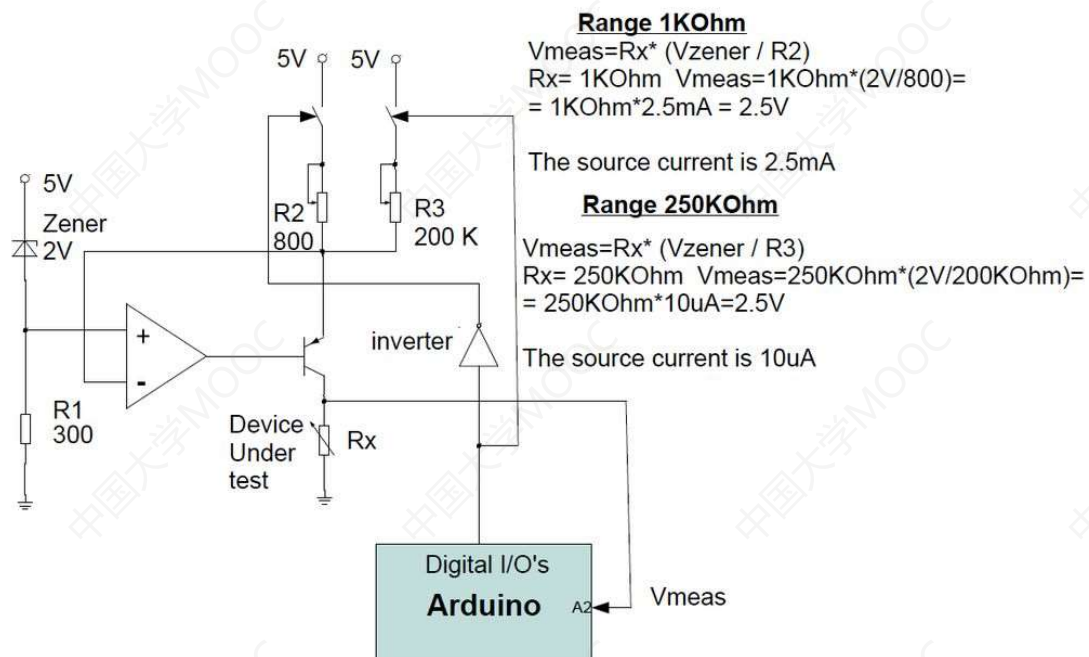


图 4.24 欧姆表的简图

“Arduino”板控制两个开关中的哪一个闭合，以这种方式决定两个电阻中的哪一个将流过电流。因此，两个可能的电流值是可能的： $10\mu\text{A}$ ， 2.5mA 。这些电流可以非常精确地调节。这样产生的电流通过被测器件(电阻、二极管、LED 和晶体管等)，在被测器件(DUT)上电压降由“Arduino”板的模拟引脚 A2 检测。

4、测量 beta 值：前面提到的产生的电流可以流过不同的器件：电阻、二极管、LED、肖特基二极管……，产生的电压降和相应的器件有关。这个电压值可以提供相应器件的信息，比如器件的电压降和器件的对应：二极管-0.4V-0.8V；肖特基二极管-0.1V-0.5V，LED（颜色不同并不一样）-1.1V-3.5V。这时使用的电流是 10uA。而如果使用 2.5mA，那么 LED 就开始发光，就能测量 LED 的功能性。测量 beta 值也是使用的 10uA，测量值可在屏幕上读出。

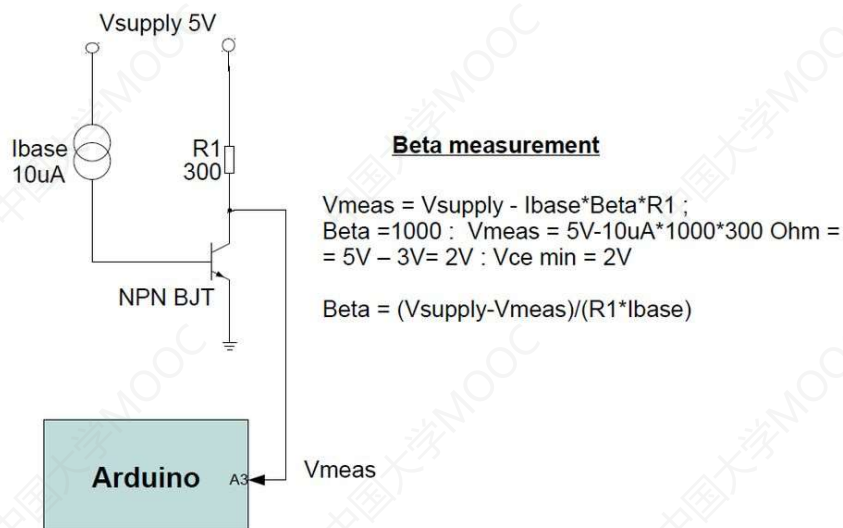


图 4.25 晶体管 beta 值测试电路

四、软件设计

程序框图如图 4.26 所示，各部分程序，请大家自己设计。

五、实验任务与要求

1、用 Arduino Uno 板设计万用表，测量数据可以用 LCD 显示模块显示。该万用表应具备以下功能：

- 具备 3 个量程的电压表：0-10V，0-30V，0-100V
- 具备 1 个量程的电流表：0-500mA
- 具备 2 个量程的欧姆表：0-1kΩ，0-250kΩ
- 二极管，LED 的连接性检测
- LED 功能性测试
- NPN 双极结型晶体管的 BETA 值测量

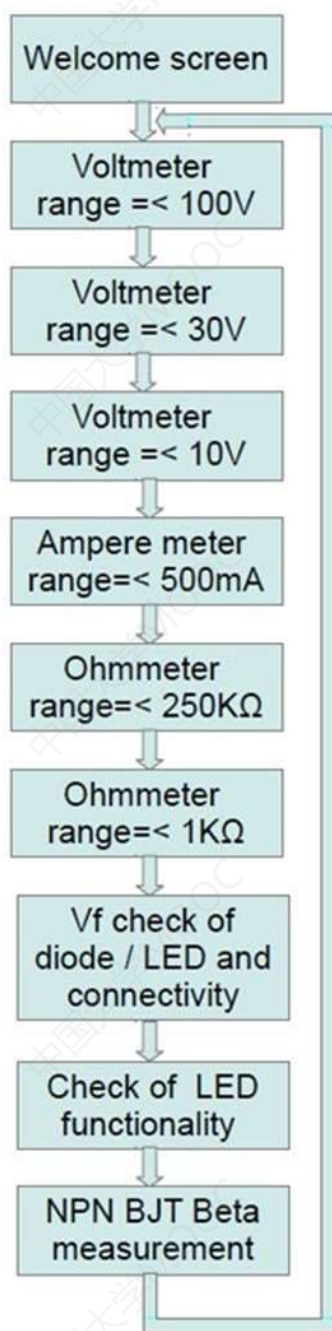


图 4.26 程序设计框图

- 2、设计的电路，完成相应器件的选择和参数计算，制作 Arduino UNO 扩展板。
- 3、编制与调试万用表的程序。
- 4、将制作的 Arduino UNO 扩展板与 Arduino UNO 板组装后，进行系统联调。
- 5、用一个标准的万用表，对组装的万用表进行校准。

4.8 Arduino 数字示波器的设计与制作

一、实验目的

- 1、学习掌握用 Arduino UNO 设计示波器；
- 2、学习掌握 PCB 电路板的设计和制作；
- 3、学习掌握 Arduino UNO 扩展板的设计与制作；
- 4、学习掌握 LCD1602 液晶显示屏的使用。

二、实验设计方案

本项目使用 Arduino UNO 设计示波器。硬件框图如图 4.27 所示，下面对各部分模块进行说明。

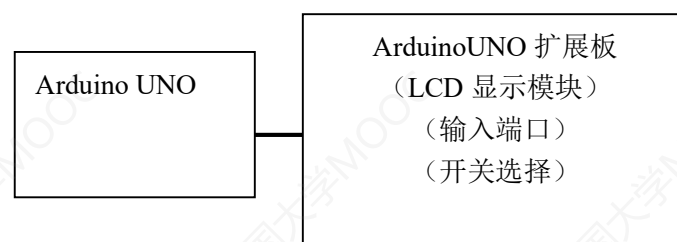


图 4.27 Arduino 数字示波器框图

LCD12864 液晶模块

12864LCD 显示块是所说的点阵液晶显示模块，就是由 128*64 个液晶显示点组成的一个 128 列*64 行的阵列，所以也就叫成了 12864。每个显示点都对应着有一位二进制数，0 表示灭，1 表示亮。存储这些点阵信息的 RAM 被称为显示数据存储器。如果要显示某个图形或汉字就是将相应的点阵信息写入到对应的存储单元中。图形或汉字的点阵信息是由自己设计，这时候问题的关键是显示点在液晶屏上的位置与其在存储器中的地址之间的关系。

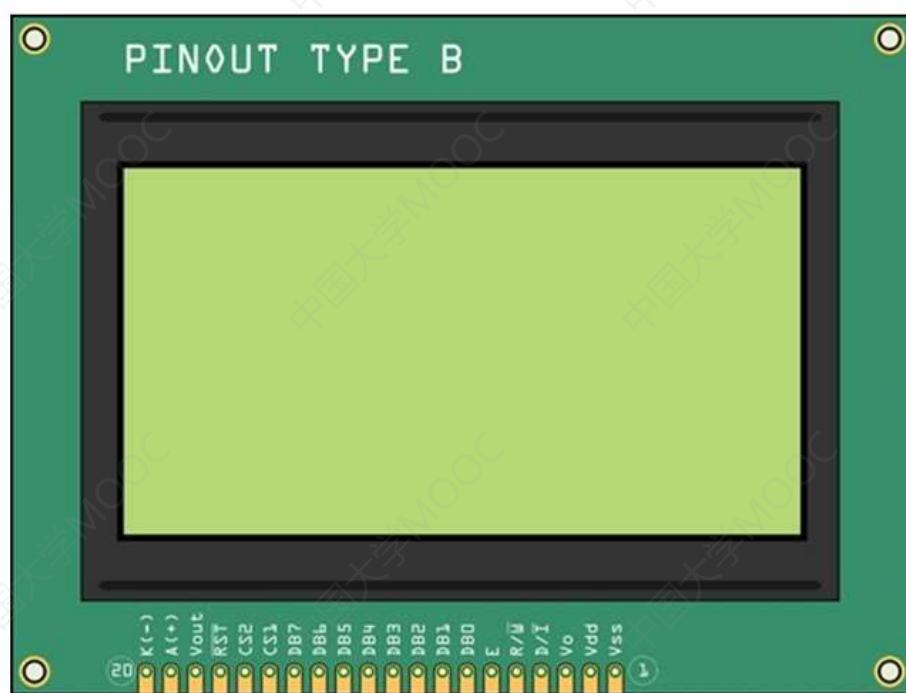


图 4.28 TS12864 液晶实物图

带中文字库的 TS12864（图 4.28）是一种具有 4 位/8 位并行、2 线或 3 线串行多种接口方式，引脚功能见图表 4-1，显示分辨率为 128×64，内置 8192 个 16*16 点汉字，和 128 个 16*8 点 ASCII 字符集.利用该模块灵活的接口方式和简单、方便的操作指令，可构成全中文人机交互图形界面。可以显示 8×4 行 16×16 点阵的汉字、也可完成图形显示、低电压低功耗是其又一显著特点。由该模块构成的液晶显示方案与同类型的图形点阵液晶显示模块相比，不论硬件电路结构或显示程序都要简洁得多，且该模块的价格也略低于相同点阵的图形液晶模块。

表 4-1 12864 液晶显示模块引脚功能表

编号	名称	引脚说明	编号	名称	引脚说明
1	VSS	电源地	11	DB4	三态数据线
2	VDD	电源正极	12	DB5	三态数据线
3	V0	对比度调整	13	DB6	三态数据线
4	RS(CS)	显示数据	14	DB7	三态数据线
5	R/W	读/写选择端 (H/L)	15	PSB	传输方式
6	E	使能信号	16	NC	悬空
7	DB0	三态数据线	17	RESET	复位端，低电平有效
8	DB1	三态数据线	18	VOUT	驱动电压输出端
9	DB2	三态数据线	19	A	背光源正极
10	DB3	三态数据线	20	K	背光源负极

由上表 4-1 所示，我们可知 DB0~DB7 为液晶显示模块的数据传输端口，引脚 1~3 为电源地、电源正极及对比度调整。

三、硬件设计

Arduino 数字示波器电路，如图 4.29 所示。

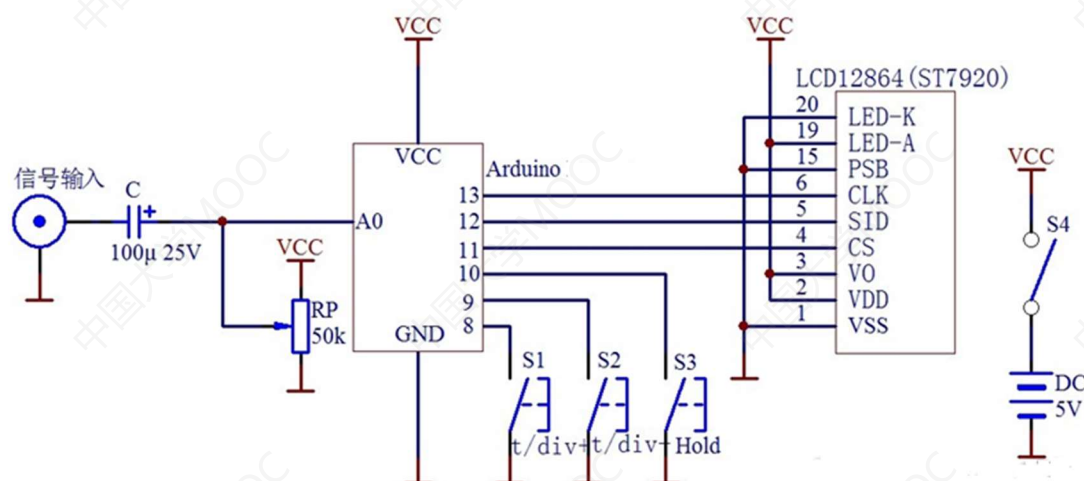


图 4.29 Arduino 数字示波器电路原理图

四、软件设计

使用 Arduino 做项目的最大优势就是其丰富的资源，同时也不需要了解太多单片机的知识。利用 LCD 的资源库 u8glib，可使编程变得简单多了，不然 LCD 的驱动会花费好多时间。

按图 4.29 搭建电路，输入下列代码，编译下载后就能实现数字示波器的基本功能。

```
/*
    arduino 数字示波器
    作者:?ntwhq@tom.com
    2013 08
*/
#include <U8glib.h> //声明库
U8GLIB_ST7920_128X64_4X u8g(13, 12, 11); // 声明液晶屏 13=sck, 12=mosi, 11=cs
int Input = A0; //声明输入引脚
int Key_add = 8; //声明按钮引脚
int Key_sub = 9;
int Key_hold = 10;
int x,y; //声明坐标
int i,i1,i2,V_min,V_max,V_mid,t,t0,t1,sta,Key=1,hold=0;
long Freq;
float Vpp;
int Y[96]; //声明信号值储存数组
int Buffer[192];
const uint8_t L[] PROGMEM = {
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x01,0xC0,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x01,0xE0,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x03,0xE0,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x03,0x70,0x3F,0x87,0xF0,0xC1,0x99,0x83,
0x07,0x80,0x00,0x00,0x00,0x00,0x00,0x00,
0x07,0x30,0x3F,0xC7,0xF8,0xC1,0x99,0xC3,
0x1F,0xE0,0x00,0x00,0x00,0x00,0x00,0x00,
0x06,0x38,0x30,0xC6,0x1C,0xC1,0x99,0xE3,
0x18,0x60,0x00,0x00,0x00,0x00,0x00,0x00,
0x0E,0x18,0x30,0xC6,0x0C,0xC1,0x99,0xF3,
0x30,0x30,0x00,0x00,0x00,0x00,0x00,0x00,
0x0F,0xFC,0x3F,0xC6,0x0C,0xC1,0x99,0xBB,
0x30,0x30,0x00,0x00,0x00,0x00,0x00,0x00,
0x1F,0xFC,0x3F,0x06,0x0C,0xC1,0x99,0x9F,
0x30,0x30,0x00,0x00,0x00,0x00,0x00,0x00,
0x18,0x0E,0x31,0x86,0x1C,0xE3,0x99,0x8F,
0x38,0x70,0x00,0x00,0x00,0x00,0x00,0x00,
0x38,0x07,0x30,0xC7,0xF8,0x7F,0x19,0x87,
0x1F,0xE0,0x00,0x00,0x00,0x00,0x00,0x00,
```

0x78,0x07,0xB0,0xE7,0xE0,0x3E,0x19,0x87,
0x07,0x80,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x82,0x00,0x10,
0x00,0x00,0x00,0x00,0x80,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x04,0x92,0x00,0x08,
0x00,0x00,0x20,0x40,0x80,0x3E,0x78,0x00,
0x00,0x00,0x00,0x00,0x02,0xA4,0x01,0xFF,
0xE0,0x7F,0xC0,0x40,0x80,0x24,0x88,0x00,
0x00,0x00,0x00,0x00,0x02,0xC4,0x02,0x00,
0x40,0x00,0x00,0x08,0x88,0x24,0x88,0x00,
0x00,0x00,0x00,0x00,0x0F,0xF7,0xE2,0x00,
0x80,0x00,0x00,0x1F,0xF8,0x3C,0x78,0x00,
0x00,0x00,0x00,0x00,0x01,0xCC,0x81,0xFF,
0x80,0x00,0x19,0xA8,0x90,0x26,0xA0,0x00,
0x00,0x00,0x00,0x00,0x02,0xB4,0x80,0x02,
0x03,0xFF,0xE0,0xA8,0x80,0x03,0x14,0x00,
0x00,0x00,0x00,0x00,0x04,0x94,0x80,0x04,
0x00,0x24,0x00,0x27,0xF8,0x7C,0xE8,0x00,
0x00,0x00,0x00,0x00,0x08,0x84,0x80,0x0C,
0x00,0x24,0x80,0x4A,0x10,0x0C,0x40,0x00,
0x00,0x00,0x00,0x00,0x01,0x34,0x87,0xFF,
0xF0,0x44,0x40,0x49,0x20,0x10,0x30,0x00,
0x00,0x00,0x00,0x00,0x07,0xE3,0x00,0x08,
0x00,0x44,0x20,0x49,0x20,0x7E,0xFE,0x00,
0x00,0x1E,0x00,0x00,0x02,0x43,0x00,0x08,
0x00,0x84,0x10,0x88,0xC0,0x22,0x88,0x00,
0x00,0x33,0x00,0x00,0x01,0xC3,0x00,0x08,
0x01,0x04,0x10,0x90,0xC0,0x22,0x88,0x00,
0x00,0x61,0x80,0x00,0x01,0xB4,0x80,0x08,
0x02,0x04,0x00,0x91,0x30,0x22,0x88,0x00,
0x00,0xC0,0x80,0x00,0x02,0x08,0x60,0x38,
0x00,0x1C,0x00,0xA6,0x1C,0x3E,0xF8,0x00,
0x00,0x80,0xC0,0x00,0x04,0x30,0x00,0x10,
0x00,0x08,0x00,0x58,0x00,0x00,0x00,0x00,
0x01,0x80,0x40,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x01,0x80,0x60,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,

0x01,0x00,0x60,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x03,0x00,0x20,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x02,0x00,0x30,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x02,0x00,0x30,0x00,0x00,0x00,0x00,0x03,
0x00,0x60,0x00,0x20,0x00,0x18,0x00,0x00,
0x06,0x00,0x10,0x00,0x00,0x00,0x00,0x06,
0x00,0x30,0x00,0x60,0x00,0x18,0x00,0x00,
0x06,0x00,0x10,0x00,0x00,0x00,0x00,0x06,
0x00,0x30,0x00,0x60,0x00,0x18,0x00,0x00,
0x04,0x00,0x10,0x00,0x00,0x00,0x00,0x06,
0x1E,0x31,0xBC,0xFE,0x38,0xDB,0xC1,0xD8,
0x04,0x00,0x18,0x00,0x00,0x00,0x00,0x0C,
0x3F,0x19,0xFE,0xFE,0x38,0xDF,0xE3,0xF8,
0x04,0x00,0x18,0x00,0x10,0x00,0x00,0x0C,
0x73,0x19,0xC6,0x63,0x39,0x9C,0x67,0x38,
0x0C,0x00,0x18,0x00,0x18,0x00,0x00,0x0C,
0x60,0x19,0x86,0x63,0x6D,0x98,0x66,0x18,
0x7F,0xFF,0xFF,0xFF,0xFE,0x00,0x00,0x0C,
0x60,0x19,0x86,0x63,0x6D,0x98,0x66,0x18,
0x0C,0x00,0x08,0x00,0x10,0x00,0x00,0x0C,
0x60,0x19,0x86,0x63,0x6D,0x98,0x66,0x18,
0x00,0x00,0x08,0x00,0x30,0x00,0x00,0x0C,
0x73,0x19,0x86,0x61,0xC7,0x18,0x67,0x38,
0x00,0x00,0x0C,0x00,0x30,0x00,0x00,0x0C,
0x3F,0x19,0x86,0x79,0xC7,0x18,0x63,0xF8,
0x00,0x00,0x0C,0x00,0x30,0x00,0x00,0x06,
0x1E,0x31,0x86,0x39,0xC7,0x18,0x61,0xD8,
0x00,0x00,0x04,0x00,0x20,0x00,0x00,0x06,
0x00,0x30,0x00,0x00,0x00,0x00,0x00,0x18,
0x00,0x00,0x04,0x00,0x20,0x00,0x00,0x06,
0x00,0x30,0x00,0x00,0x00,0x00,0x00,0x18,
0x00,0x00,0x06,0x00,0x60,0x00,0x00,0x03,
0x00,0x60,0x00,0x00,0x00,0x00,0x00,0x18,
0x00,0x00,0x06,0x00,0x60,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x02,0x00,0x40,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x03,0x00,0xC0,0x00,0x00,0x00,
0x00,0x00,0x01,0xC7,0x04,0x70,0x38,0xE0,
0x00,0x00,0x03,0x00,0x80,0x00,0x00,0x00,
0x00,0x00,0x02,0x28,0x8C,0x88,0x45,0x10,


```

0x00,0x00,0x01,0x01,0x80,0x00,0x00,0x00,
0x00,0x00,0x00,0x28,0x94,0x08,0x45,0x10,
0x00,0x00,0x01,0x81,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x48,0x84,0x30,0x44,0xE0,
0x00,0x00,0x00,0xC3,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x88,0x84,0x08,0x45,0x10,
0x00,0x00,0x00,0x66,0x00,0x00,0x00,0x00,
0x00,0x00,0x01,0x08,0x84,0x88,0x45,0x10,
0x00,0x00,0x00,0x3C,0x00,0x00,0x00,0x00,
0x00,0x00,0x03,0xE7,0x04,0x70,0x38,0xE0,
0x00,0x00,0x00,0x18,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00
};

```

```

void setup( )
{
    pinMode(Key_add,INPUT);
    digitalWrite(Key_add,HIGH);
    pinMode(Key_sub,INPUT);
    digitalWrite(Key_sub,HIGH);
    pinMode(Key_hold,INPUT);
    digitalWrite(Key_hold,HIGH);
    ADMUX=0xe0;
    ADCSRA=0xe2;
    u8g.setFont(u8g_font_5x7);
    u8g.firstPage();
    do {
        u8g.drawBitmapP( 0, 0, 16, 64, L);
    } while( u8g.nextPage() );
    delay(2000);
}

```

```

void loop( )
{
    sample( );
    Measure( );
    Transform( );
    Key_scan( );
    if(hold==0)
    {

```

```

    u8g.firstPage( );
    do
    {
        draw( );
    }
    while( u8g.nextPage( ));
}

void sample( )
{
    for(i = 0;i < 192;i++)
    {
        Buffer[i] = ADCH;
        switch(Key)
        {
            case 1:
                break;
            case 2:
                delayMicroseconds(4);
                break;
            case 3:
                delayMicroseconds(10);
                break;
            case 4:
                delayMicroseconds(23);
                break;
            case 5:
                delayMicroseconds(60);
                break;
            case 6:
                delayMicroseconds(123);
                break;
            case 7:
                delayMicroseconds(248);
                break;
            case 8:
                delayMicroseconds(623);
                break;
            case 9:
                delayMicroseconds(1247);
                break;
            default:break;
        }
    }
}

```

```

}

void Measure()
{
    V_max=Buffer[0];
    V_min=Buffer[0];
    for(i=0;i<192;i++)
    {
        if(Buffer[i]>V_max)
            V_max=Buffer[i];
        if(Buffer[i]<V_min)
            V_min=Buffer[i];
    }
    V_mid=(V_max+V_min)/2;
    Vpp=(V_max-V_min)*1.1/255;
    for(i=0;i<97;i++)
    {
        if(Buffer[i]<V_mid&&Buffer[i+1]>=V_mid)
        {
            i1=i;
            break;
        }
    }
    for(i=i1+1;i<98+i1;i++)
    {
        if(Buffer[i]<V_mid&&Buffer[i+1]>=V_mid)
        {
            i2=i;
            break;
        }
    }
    t=i2-i1;
    if(t>0)
        Freq=8000/t;
    else
        Freq=0;
}

```

```

void Transform()
{
    for(sta=0;sta<96;sta++)
    {
        if(Buffer[sta]<128&&Buffer[sta+2]>128)
            break;
    }
}

```

```

    }
    for(i = 0;i < 96;i++)
        Y[i] = 63-(Buffer[i+sta]>>2);
}

void draw( )
{
    for(x = 0;x < 95;x++)
        u8g.drawLine(x,Y[x],x,Y[x+1]); //画线

    //画边框
    u8g.drawFrame(0,0,97,64);
    // 画坐标轴
    u8g.drawLine(48,0,48,63);
    u8g.drawLine(0,32,96,32);
    for(x=0;x<96;x+=8)
        u8g.drawLine(x,31,x,33);
    for(y=0;y<64;y+=8)
        u8g.drawLine(47,y,49,y);
    //画网格
    for(x=8;x<96;x+=8)
    {
        for(y=8;y<64;y+=8)
            u8g.drawPixel(x,y);
    }
    //显示参数
    u8g.drawStr(98,7,"MS/div");
    u8g.drawStr(98,23,"V/div");
    u8g.drawStr(98,30,"0.324");
    u8g.drawStr(98,40,"Vpp");
    u8g.setPrintPos( 98, 47);
    u8g.print(Vpp);
    u8g.drawStr(118,47,"V");
    u8g.drawStr(98,55,"F(HZ)");
    switch(Key)
    {
        case 1:
            u8g.drawStr(98,14,"0.02");
            u8g.setPrintPos( 98, 62);
            u8g.print(Freq*50);
            break;
        case 2:
            u8g.drawStr(98,14,"0.05");
            u8g.setPrintPos( 98, 62);

```

```

        u8g.print(Freq*20);
        break;
        case 3:
            u8g.drawStr(98,14," 0.1");
            u8g.setPrintPos( 98, 62);
            u8g.print(Freq*10);
            break;
        case 4:
            u8g.drawStr(98,14," 0.2");
            u8g.setPrintPos( 98, 62);
            u8g.print(Freq*5);
            break;
        case 5:
            u8g.drawStr(98,14," 0.5");
            u8g.setPrintPos( 98, 62);
            u8g.print(Freq*2);
            break;
        case 6:
            u8g.drawStr(98,14," 1");
            u8g.setPrintPos( 98, 62);
            u8g.print(Freq);
            break;
        case 7:
            u8g.drawStr(98,14," 2");
            u8g.setPrintPos( 98, 62);
            u8g.print(Freq/2);
            break;
        case 8:
            u8g.drawStr(98,14," 5");
            u8g.setPrintPos( 98, 62);
            u8g.print(Freq/5);
            break;
        case 9:
            u8g.drawStr(98,14," 10");
            u8g.setPrintPos( 98, 62);
            u8g.print(Freq/10);
            break;
        default:break;
    }
}

//键盘扫描
void Key_scan()
{

```

```

if(digitalRead(Key_add)==LOW)
{
    while(digitalRead(Key_add)==LOW);
    Key++;
    if(Key==10)
    Key=9;
    delay(10);
}
if(digitalRead(Key_sub)==LOW)
{
    while(digitalRead(Key_sub)==LOW);
    Key--;
    if(Key==0)
    Key=1;
    delay(10);
}
if(digitalRead(Key_hold)==LOW)
{
    while(digitalRead(Key_hold)==LOW);
    hold=~hold;
    delay(10);
}
}

```

上述程序 ADC 使用片内 1.1V 基准电压，如果要使用片外 5V 基准电压，应将程序中
 ADMUX=0xe0; 改为: ADMUX=0x60;
 Vpp=(V_max-V_min)*1.1/255;改为: Vpp=(V_max-V_min)*5/255;

五、实验任务与要求

1、用 Arduino UNO 设计示波器，要求实现功能和参数：

频率响应：10Hz-50kHz

电源： 5V

LCD 液晶屏： 128x64（ST7920）

测量显示区：96x64

信息显示区：32x64，显示测试信号的频率、Vpp 等内容

同步方式：上升沿触发

扫描速度：0.02ms/div~10ms/div,按 1-2-5 进位分九档

Hold 功能：冻结显示波形和参数

2、设计的电路，完成相应器件的选择和参数计算，制作 Arduino UNO 扩展板。

3、编制与调试示波器的程序。

4、将制作的 Arduino UNO 扩展板与 Arduino UNO 板组装后，进行系统联调。

1602LCD 主要技术参数：
显示容量为 16×2 个字符；
芯片工作电压为 $4.5 \sim 5.5\text{V}$ ；
工作电流为 2.0mA (5.0V)；
模块最佳工作电压为 5.0V ；
字符尺寸为 2.95×4.35 (W×H) mm。

1602 采用标准的 16 脚接口, 其中:

第 1 脚: VSS 为地电源

第2脚: VDD 接 5V 正电源

第3脚：V0为液晶显示器对比度调整端，接正电源时对比度最弱，接地电源时对比度最高，对比度过高时会产生“鬼影”，使用时可以通过一个10K的电位器调整对比度

第4脚：RS为寄存器选择，高电平时选择数据寄存器、低电平时选择指令寄存器。

第 5 脚: R/W 为读写信号线, 高电平时进行读操作, 低电平时进行写操作。当 RS 和 RW 共同为低电平时可以写入指令或者显示地址, 当 RS 为低电平 RW 为高电平时可以读信号, 当 RS 为高电平 RW 为低电平时可以写入数据。

第6脚：E端为使能端，当E端由高电平跳变成低电平时，液晶模块执行命令。

第 7~14 脚: D0~D7 为 8 位双向数据线。

第 15 脚: 背光电源正极

第 16 脚：背光电源负极

LCD 可按 4 位模式连接, 如图 1 所示。

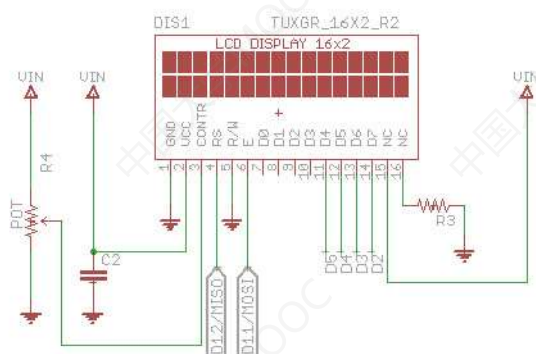


图 F.1 LCD 4 位连接模式

程序模版如下：

```
#include <LiquidCrystal.h>

LiquidCrystal iLCD (12, 11, 5, 4, 3, 2);

void setup ( )
{
    iLCD.begin (16, 2 );
    // your code here...
}

void loop ()
{
    // your code here ...
}
```

在 Arduino 的安装目录下\libraries\LiquidCrystal 可以查看到函数的原型

LiquidCrystal()——定义你的 LCD 的接口：各个引脚连接的 I/O 口编号，格式为 LiquidCrystal(rs, enable, d4, d5, d6, d7) //4 位连接模式

LiquidCrystal(rs, rw, enable, d4, d5, d6, d7) //4 位连接模式

LiquidCrystal(rs, enable, d0, d1, d2, d3, d4, d5, d6, d7) //8 位连接模式

LiquidCrystal(rs, rw, enable, d0, d1, d2, d3, d4, d5, d6, d7) //8 位连接模式

begin()——定义 LCD 的长宽（n 列×n 行），格式 lcd.begin(cols, rows)

clear()——清空 LCD，格式 lcd.clear()

home()——把光标移回左上角，即从头开始输出，格式 lcd.home()

setCursor()——移动光标到特定位置，格式 lcd.setCursor(col, row)

write()——在屏幕上显示内容（必须是一个变量，如"Serial.read()"），格式 lcd.write(data)

print()——在屏幕上显示内容（字母、字符串，等等），格式 lcd.print(data)

lcd.print(data, BASE)

cursor()——显示光标（一条下划线），格式 lcd.cursor()

noCursor()——隐藏光标，格式 lcd.noCursor()

blink()——闪烁光标，格式 lcd.blink()

noBlink()——光标停止闪烁，格式 lcd.noBlink()

display()——（在使用 noDisplay()函数关闭显示后）打开显示（并恢复原来内容），格式 lcd.display()

noDisplay()——关闭显示，但不会丢失原来显示的内容，格式为 lcd.noDisplay()

scrollDisplayLeft()——把显示的内容向左滚动一格，格式 lcd.scrollDisplayLeft()

scrollDisplayRight()——把显示的内容向右滚动一格，格式为 lcd.scrollDisplayRight()

autoscroll()——打开自动滚动，这使每个新的字符出现后，原有的字符都移动一格：如果字符一开始从左到右（默认），那么就往左移动一格，否则就向右移动，格式 lcd.autoscroll()

noAutoscroll()——关闭自动滚动，格式 lcd.noAutoscroll()

leftToRight()——从左往右显示，也就是说显示的字符会从左往右排列（默认），但屏幕上已经有的字符不受影响，格式 lcd.leftToRight()

rightToLeft()——从右往左显示，格式 lcd.rightToLeft()

附录 2

图 F.2 示出了当旋转编码器工作时输出 A 和 B 的信号是如何变化的。当顺时针旋转旋钮时，脉冲会变化，并且会在图中从左向右移动；当逆时针旋转旋钮时，脉冲会在图中从右向左移动。

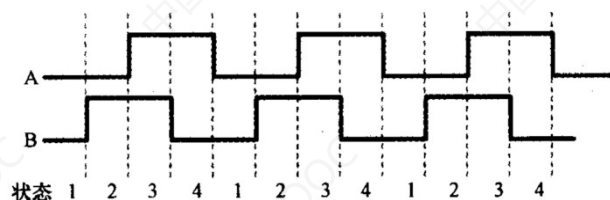


图 F.2 旋转编码器输出的脉冲波形

因此，如果 A 和 B 都是低电平，然后 B 变为高电平（从状态 1 到状态 2），这表明旋钮已经顺时针旋转；如果 A 低、B 高，然后 A 由低变高（从状态 2 到状态 3）也同样表明旋钮顺时针旋转；如果 A 高、B 低，然后 B 变高，已经从状态 4 到状态 3，表明旋钮逆时针旋转。

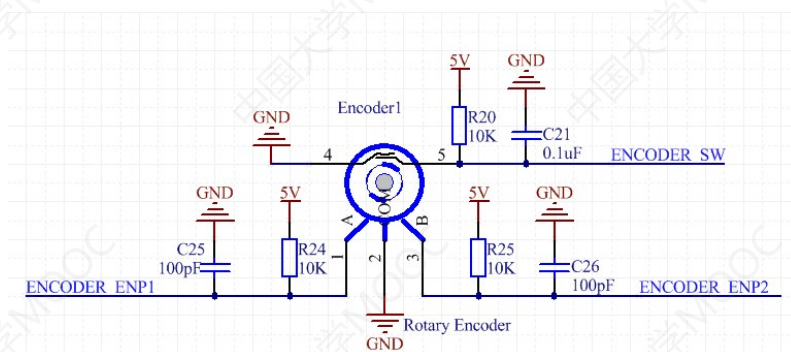


图 F.3 旋转编码器接线图

图 F.3 为 EC11 旋转编码器接线图，1、3 脚为 A、B 信号，4、5 脚为开关，2 脚接地。